

## Indice

- <https://docs.google.com/document/d/1oqV-Ig29J8W2rwN4uC9I6tXjWHIWfnKaB7-AI/MephDg/edit?tab=t.u64iih4gwmuj#heading=h.skdjucz5ls20> Lab 01
- <https://docs.google.com/document/d/1oqV-Ig29J8W2rwN4uC9I6tXjWHIWfnKaB7-AI/MephDg/edit?tab=t.u64iih4gwmuj#heading=h.ui8jvlcikp7f> Lab 02
- <https://docs.google.com/document/d/1oqV-Ig29J8W2rwN4uC9I6tXjWHIWfnKaB7-AI/MephDg/edit?tab=t.u64iih4gwmuj#heading=h.ysfx6bmdep43> Lab 03
- <https://docs.google.com/document/d/1oqV-Ig29J8W2rwN4uC9I6tXjWHIWfnKaB7-AI/MephDg/edit?tab=t.u64iih4gwmuj#heading=h.165g1bo7c15z> Lab 04
- <https://docs.google.com/document/d/1oqV-Ig29J8W2rwN4uC9I6tXjWHIWfnKaB7-AI/MephDg/edit?tab=t.u64iih4gwmuj#heading=h.c781sioabhog> Lab 05
- <https://docs.google.com/document/d/1oqV-Ig29J8W2rwN4uC9I6tXjWHIWfnKaB7-AI/MephDg/edit?tab=t.u64iih4gwmuj#heading=h.tly3k7r0f8u> Lab 06
- <https://docs.google.com/document/d/1oqV-Ig29J8W2rwN4uC9I6tXjWHIWfnKaB7-AI/MephDg/edit?tab=t.u64iih4gwmuj#heading=h.281myjsr53zr> lab 07
- <https://docs.google.com/document/d/1oqV-Ig29J8W2rwN4uC9I6tXjWHIWfnKaB7-AI/MephDg/edit?tab=t.u64iih4gwmuj#heading=h.c0xc0snjvafh> lab 08-09

## Lab 01 - Green IT: Smart Working e Smart Grid

Green IT: Smart Working e Smart Grid

### Step 1 - Calcolo consumo attuale data center

Consumi energetici — confronto annuale

**FictaCorp** · 200 dipendenti · 10 rack server · Ufficio 80 000 kWh/anno

| Scenario                                             | Data center<br>(kWh) | Ufficio<br>(kWh) | Totale<br>(kWh) | Risparmio vs<br>baseline | Stato    |
|------------------------------------------------------|----------------------|------------------|-----------------|--------------------------|----------|
| <b>On-premise attuale</b><br>PUE 1,8 · nessuno<br>SW | 788 400              | 80 000           | <b>868 400</b>  | —                        | Baseline |

|                                                                                   |         |        |                |             |      |
|-----------------------------------------------------------------------------------|---------|--------|----------------|-------------|------|
| <b>Cloud AWS</b><br>PUE 1,2 · right-sizing<br>-30% · nessuno SW                   | 367 920 | 80 000 | <b>447 920</b> | 420 480 kWh | -48% |
| <b>Cloud + Smart Working 60%</b><br>PUE 1,2 · right-sizing<br>-30% · ufficio -40% | 367 920 | 32 000 | <b>399 920</b> | 468 480 kWh | -54% |

## Step 2 - Stima consumo equivalente su AWS

### Consumi energetici — confronto trimestrale (stagionale)

Valori = annuale ÷ 4 · Trimestre tipo (90,25 giorni medi)

| Scenario                         | Data center (kWh) | Ufficio (kWh) | Totale (kWh)   | Risparmio trimestrale | Stato           |
|----------------------------------|-------------------|---------------|----------------|-----------------------|-----------------|
| <b>On-premise attuale</b>        | 197 100           | 20 000        | <b>217 100</b> | —                     | <b>Baseline</b> |
| <b>Cloud AWS</b>                 | 91 980            | 20 000        | <b>111 980</b> | 105 120 kWh           | -48%            |
| <b>Cloud + Smart Working 60%</b> | 91 980            | 8 000         | <b>99 980</b>  | 117 120 kWh           | -54%            |

## Step 3 - Impatto dello smart working

### Risparmio CO2

| Transizione                  | kWh risparmiati/anno | kWh risparmiati/trim. | CO <sub>2</sub> evitata/anno (t) | CO <sub>2</sub> evitata/trim. (t) |
|------------------------------|----------------------|-----------------------|----------------------------------|-----------------------------------|
| On-premise → Cloud           | 420 480              | 105 120               | 105,1                            | 26,3                              |
| Solo Smart Working (ufficio) | 48 000               | 12 000                | 12,0                             | 3,0                               |
| On-premise → Cloud + SW      | 468 480              | 117 120               | 117,1                            | 29,3                              |

**Schema AWS smart grid**

| Layer             | Servizio AWS                      | Protocollo / tipo | Ruolo nella smart grid                                                                                                                                                                     | Input / Output principali                                                          |
|-------------------|-----------------------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Campo</b>      | Smart meter<br>Dispositivo IoT    | MQTT / HTTP       | Misura i consumi energetici in tempo reale presso utenze, rack server e impianti. Invia letture periodiche (ogni 15 min o meno) verso il cloud.                                            | IN: alimentazione elettrica<br>OUT: kWh, tensione, corrente → IoT Core             |
| <b>Storage</b>    | Amazon S3<br>Object storage       | REST / S3 API     | <b>Archivia la serie storica completa di tutte le letture dei sensori in formato Parquet o CSV. Funge da data lake per il training dei modelli ML e per audit e compliance.</b>            | IN: record da Kinesis<br>OUT: dataset → SageMaker, QuickSight                      |
| <b>Processing</b> | AWS Lambda<br>Serverless function | Event-driven      | <b>Esegue logica di elaborazione senza server dedicato: normalizzazione dei dati, generazione di alert su soglie di consumo, chiamate agli endpoint SageMaker per inferenza immediata.</b> | IN: eventi da Kinesis / IoT Core<br>OUT: alert, dati normalizzati → S3, SNS        |
| <b>Ingestione</b> | AWS IoT Core<br>Managed broker    | MQTT / WebSocket  | Riceve i messaggi dai sensori distribuiti, gestisce l'autenticazione dei dispositivi e smista i dati verso Kinesis o Lambda tramite regole configurabili.                                  | IN: messaggi MQTT dai sensori<br>OUT: topic → Kinesis Data Streams                 |
| <b>Ingestione</b> | Amazon Kinesis<br>Data Streams    | Stream real-time  | <b>Processa lo streaming continuo di dati energetici con bassa latenza. Rileva picchi di carico e variazioni di tensione in tempo reale prima che i dati vengano archiviati.</b>           | IN: stream da IoT Core<br>OUT: record → S3, Lambda                                 |
| <b>Analytics</b>  | Amazon SageMaker<br>ML platform   | REST endpoint     | Addestra modelli di previsione dei carichi (DeepAR, LSTNet) sui dati storici in S3 e espone endpoint per inferenza. Rileva anche anomalie (RandomCutForest) sui consumi misurati.          | IN: dataset storico da S3<br>OUT: forecast kWh, anomaly score → QuickSight, Lambda |

**Gruppo 2:** *Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi*

|                  |                                |               |                                                                                                                                                                                                 |                                                             |
|------------------|--------------------------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Analytics</b> | Amazon QuickSight BI dashboard | HTTPS / embed | <b>Visualizza in dashboard interattive i consumi in tempo reale, le previsioni SageMaker e i KPI di sostenibilità (kWh risparmiati, CO<sub>2</sub> evitata) per il management di FictaCorp.</b> | IN: dati da S3, SageMaker<br>OUT: dashboard → utenti finali |
|------------------|--------------------------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|

## Lab 02 - Green IT, Economia Circolare e Intro Serverless

### Step 1 - Classificazione RAEE

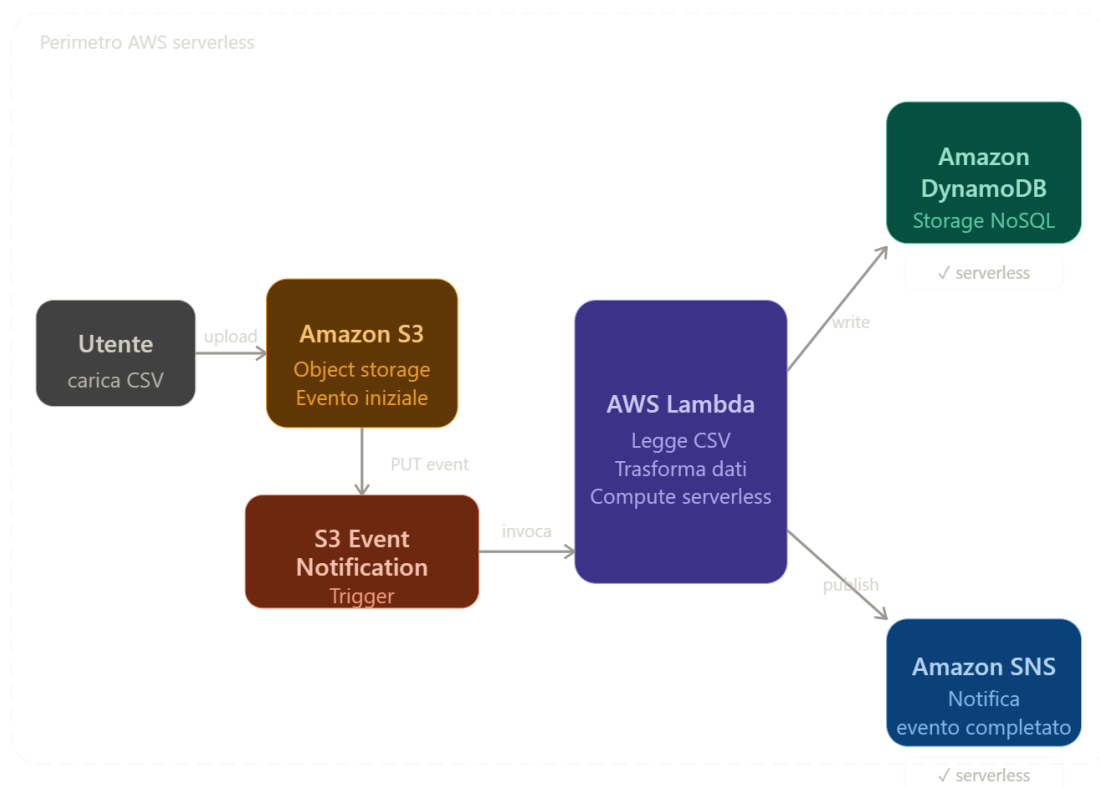
| Dispositivo                           | Quantità | Categoria RAEE                                      | Azione Consigliata        | Normativa Applicabile             |
|---------------------------------------|----------|-----------------------------------------------------|---------------------------|-----------------------------------|
| Laptop (funzionanti)                  | 50       | R3 – Apparecchiature IT e Telecomunicazioni         | Refurbishment / Donazione | Art. 3(1)(a), All. III – Cat. 3   |
| Monitor LCD (non funzionanti)         | 30       | R3 – Apparecchiature IT e Telecomunicazioni         | Riciclo Certificato       | Art. 3(1)(a), All. III – Cat. 3   |
| Server Rack (obsoleti ma funzionanti) | 20       | R3 – Apparecchiature IT e Telecomunicazioni         | Refurbishment             | Art. 3(1)(a), All. III – Cat. 3   |
| Stampanti Laser (miste)               | 30       | R3 – Apparecchiature IT (Grandi e Piccoli)          | Riciclo Certificato       | Art. 3(1)(a), All. III – Cat. 3/5 |
| Smartphone aziendali (funzionanti)    | 20       | R4 – Piccoli Apparecchi Elettronici                 | Donazione / Refurbishment | Art. 3(1)(a), All. III – Cat. 4   |
| TOTALE DISPOSITIVI                    | 150      | Direttiva EU 2012/19/UE (RAEE) – Economia Circolare |                           |                                   |

#### Legenda Azioni

## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

- **Refurbishment / Donazione** Dispositivi funzionanti ricondizionati e ceduti a enti no-profit, scuole o rivenditori.
- **Riciclo Certificato** Dispositivi non funzionanti o non refurbishable avviati a impianti RAEE autorizzati per il recupero di materie prime seconde.
- **Cat. R3 / R4** Categorie RAEE ai sensi dell'Allegato III della Direttiva 2012/19/UE: R3 = IT e Telecomunicazioni; R4 = Piccoli apparecchi elettronici.

## Step 2 – POC: Elaborazione CSV con AWS Lambda, S3, DynamoDB e SNS



**Scenario:** Un utente carica un file CSV su Amazon S3. L'evento scatena automaticamente una pipeline serverless che elabora, trasforma e persiste i dati su DynamoDB, inviando infine una notifica via SNS. Nessun server viene gestito in nessuna fase del processo.

**Paradigma:** Event-Driven Architecture (EDA) su cloud AWS. Ogni componente è managed/serverless: zero server da provisioning, zero patching, billing basato sull'uso effettivo.

## Classificazione dei componenti

| Componente                   | Ruolo nell'architettura                                                                                                                                                                                | Serverless ? | Motivazione serverless                                                                                                                                                                                       |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Amazon S3</b>             | Archiviazione del file CSV caricato dall'utente. Rappresenta il punto di ingresso (evento iniziale) dell'intera pipeline di elaborazione dati.                                                         | <b>Sì</b>    | Servizio di object storage completamente gestito da AWS: nessun server da provisioning, scaling automatico, pagamento per GB effettivamente utilizzato.                                                      |
| <b>S3 Event Notification</b> | Agisce da trigger automatico: non appena un file viene caricato su S3, notifica immediatamente AWS Lambda per avviare l'elaborazione, senza polling manuale.                                           | <b>Sì</b>    | Funzionalità nativa di S3 che opera senza risorse di calcolo dedicate. Non richiede infrastruttura aggiuntiva né configurazione di server di messaggistica.                                                  |
| <b>AWS Lambda</b>            | Cuore computazionale della pipeline: legge il file CSV da S3, applica le trasformazioni sui dati (parsing, validazione, mapping), quindi scrive i risultati su DynamoDB e pubblica la notifica su SNS. | <b>Sì</b>    | Paradigma Function-as-a-Service (FaaS) per eccellenza: nessun server da gestire, esecuzione on-demand, billing al millisecondo di esecuzione. Scala automaticamente da 0 a migliaia di esecuzioni parallele. |
| <b>Amazon DynamoDB</b>       | Database NoSQL gestito in cui Lambda persiste i dati trasformati. Offre accesso a bassa latenza e alta disponibilità per le letture successive da parte delle applicazioni EcoTech.                    | <b>Sì</b>    | Database completamente gestito (serverless in modalità on-demand): nessuna gestione di cluster, patching automatico, capacità che si adatta al traffico senza pre-provisioning.                              |

## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

|                                                 |                                                                                                                                                                                              |              |                                                                                                                                                                                        |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Amazon SNS (Simple Notification Service)</b> | Servizio di notifica publish/subscribe: Lambda pubblica un messaggio su SNS al termine dell'elaborazione. I subscriber (email, Lambda, SQS, HTTP endpoint) ricevono l'evento in tempo reale. | <b>Sì</b>    | Servizio di messaggistica completamente gestito: nessun broker da amministrare, scalabilità automatica, alta disponibilità multi-AZ nativa. Si paga per numero di messaggi pubblicati. |
| <b>TOTALE COMPONENTI SERVERLESS</b>             |                                                                                                                                                                                              | <b>5 / 5</b> | <i>Architettura 100% serverless — zero server da gestire</i>                                                                                                                           |

## Flusso di esecuzione (step-by-step)

|                              |                                                                                                                          |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>1. Caricamento</b>        | L'utente (o un sistema) carica un file CSV sul bucket Amazon S3 designato.                                               |
| <b>2. Trigger automatico</b> | S3 emette un evento di tipo s3:ObjectCreated che attiva automaticamente la funzione Lambda associata.                    |
| <b>3. Elaborazione</b>       | Lambda scarica il CSV da S3, ne fa il parsing, applica le trasformazioni business e prepara i record per la persistenza. |
| <b>4. Persistenza</b>        | I dati trasformati vengono scritti su Amazon DynamoDB in modalità batch o singola voce.                                  |
| <b>5. Notifica</b>           | Lambda pubblica un messaggio su Amazon SNS per notificare i sistemi a valle (es. applicazioni, alert email, code SQS).   |

## Vantaggi chiave dell'approccio serverless

**Costo zero a riposo:** Non vengono addebitati costi quando nessun file viene caricato. Lambda, DynamoDB (on-demand) e SNS fatturano solo per le operazioni effettuate.

**Scalabilità automatica:** La pipeline scala da 0 a migliaia di esecuzioni parallele in modo trasparente, senza intervento del team IT.



**Riduzione overhead operativo:** Nessun server da patching, nessun cluster da monitorare: il team può concentrarsi sulla logica di business del CSV.

**Time-to-market rapido:** Il POC può essere deployato in poche ore tramite AWS SAM o CDK, validando il paradigma serverless per EcoTech.

## Step 3 – EC2 t3.medium Always-On vs AWS Lambda (Pay-Per-Use) + Principi Green IT

**Obiettivo:** Quantificare il divario di costo tra un server EC2 tradizionale sempre acceso (worst case idle) e una funzione AWS Lambda per lo stesso carico di lavoro, collegando il risparmio ai principi del Green IT e dell'Economia Circolare.

**Ipotesi di base:** Regione AWS us-east-1 · EC2 t3.medium on-demand · Lambda 256 MB RAM · 100.000 invocazioni/mese · durata media 500 ms · prezzi AWS 2025.

### 1. Confronto diretto dei costi annuali

| Scenario                                     | Dettaglio del calcolo                                                                                                                                                                              | Costo mensile  | Costo annuo     |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|-----------------|
| <b>EC2 t3.medium (Always-On)</b>             | $\$0.0416/\text{ora} \times 24 \text{ h} \times 365 \text{ gg} = 8.760 \text{ ore/anno}$<br>Costo sostenuto anche a carico idle (0% utilizzo).<br>Fatturazione continua indipendente dal traffico. | <b>\$30.37</b> | <b>\$364.42</b> |
| <b>AWS Lambda (Pay-Per-Use)</b>              | $100.000 \text{ inv./mese} \times 0,5\text{s} \times 0,25 \text{ GB} = 12.500 \text{ GB-s}$<br>Free tier AWS: 400.000 GB-s/mese inclusi. Costo effettivo: $\sim \$0.0000166667$ per GB-s.          | <b>\$0.21</b>  | <b>\$2.52</b>   |
| <b>RISPARMIO ANNUALE</b> (\$364.42 – \$2.52) |                                                                                                                                                                                                    | <b>–99.3%</b>  | <b>\$361.90</b> |

### 2. Dettaglio calcolo AWS Lambda

La stima considera i parametri effettivi del carico di lavoro del POC EcoTech. Il costo Lambda risulta quasi interamente coperto dal free tier AWS, che include 1.000.000 di richieste/mese e 400.000 GB-s/mese.

**Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi**

| Parametro Lambda        | Valore       | Unità        | Note                                                   |
|-------------------------|--------------|--------------|--------------------------------------------------------|
| Invocazioni mensili     | 100.000      | inv./mese    | Carico stimato<br>POC EcoTech                          |
| Durata per invocazione  | 500          | ms (0,5s)    | Lettura CSV +<br>trasformazione<br>+ write<br>DynamoDB |
| Memoria allocata        | 256          | MB (0,25 GB) | Sufficiente per<br>elaborazione<br>CSV standard        |
| GB-secondi mensili      | 12.500       | GB-s/mese    | $100.000 \times 0,5s$<br>$\times 0,25 \text{ GB}$      |
| Free tier AWS Lambda    | 400.000      | GB-s/mese    | Inclusi<br>gratuitamente<br>— carico POC<br>coperto    |
| Costo eccedenza         | \$0.00001667 | per GB-s     | Pricing<br>us-east-1<br>(2025)                         |
| Costo mensile effettivo | ~\$0.21      | USD/mese     | Richieste +<br>compute (dopo<br>free tier)             |
| Costo annuo             | \$2.52       | USD/anno     | $\$0.21 \times 12$<br>mesi                             |

**Gruppo 2:** *Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi*

### 3. Connessione ai principi Green IT

Il risparmio economico del 99,3% non è solo un vantaggio finanziario: riflette direttamente una riduzione proporzionale del consumo energetico e dell'impatto ambientale. L'architettura serverless è un'implementazione pratica dei principi Green IT e dell'Economia Circolare applicati al software.

| Principio Green IT               | Come si applica al paradigma serverless                                                                                                          | Beneficio concreto per EcoTech                                                          |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Efficienza energetica</b>     | Lambda esegue codice solo quando necessario. Un EC2 always-on consuma energia costante anche con CPU allo 0%. Lambda azzera il consumo a riposo. | Eliminazione del consumo idle stimato: ~350 kWh/anno per singolo t3.medium risparmiati. |
| <b>Pay-as-you-go</b>             | Le risorse di calcolo vengono allocate e fatturate solo per i millisecondi di effettiva esecuzione, allineando costo e consumo reale.            | Costo ridotto da \$364.42 a \$2.52/anno (-99.3%) per lo stesso carico di lavoro.        |
| <b>Riduzione hardware fisico</b> | L'architettura serverless elimina la necessità di server fisici dedicati, riducendo la produzione di RAEE (coerente con lo Step 1 del progetto). | Nessun server fisico aggiuntivo da acquistare, gestire o dismettere per il POC.         |
| <b>Scalabilità sostenibile</b>   | Lambda scala automaticamente da 0 istanze al picco richiesto. Non vengono pre-allocated risorse in eccesso per gestire picchi ipotetici.         | Over-provisioning zero: nessuna risorsa sprecata in attesa di traffico futuro.          |

|                                 |                                                                                                                                                                |                                                                                                           |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Carbon footprint ridotta</b> | AWS ottimizza l'utilizzo dei data center condividendo l'infrastruttura tra milioni di clienti. L'efficienza aggregata è superiore a qualsiasi server dedicato. | EcoTech beneficia dei programmi AWS di energia rinnovabile (100% renewable energy commitment entro 2025). |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|

## 4. Sintesi esecutiva

**Risparmio annuo assoluto:**  $\$364.42 - \$2.52 = \$361.90$  / anno per singola pipeline di elaborazione CSV.

**Risparmio percentuale:** 99,3% — AWS Lambda costa meno di 1 centesimo per ogni dollaro speso su EC2 always-on.

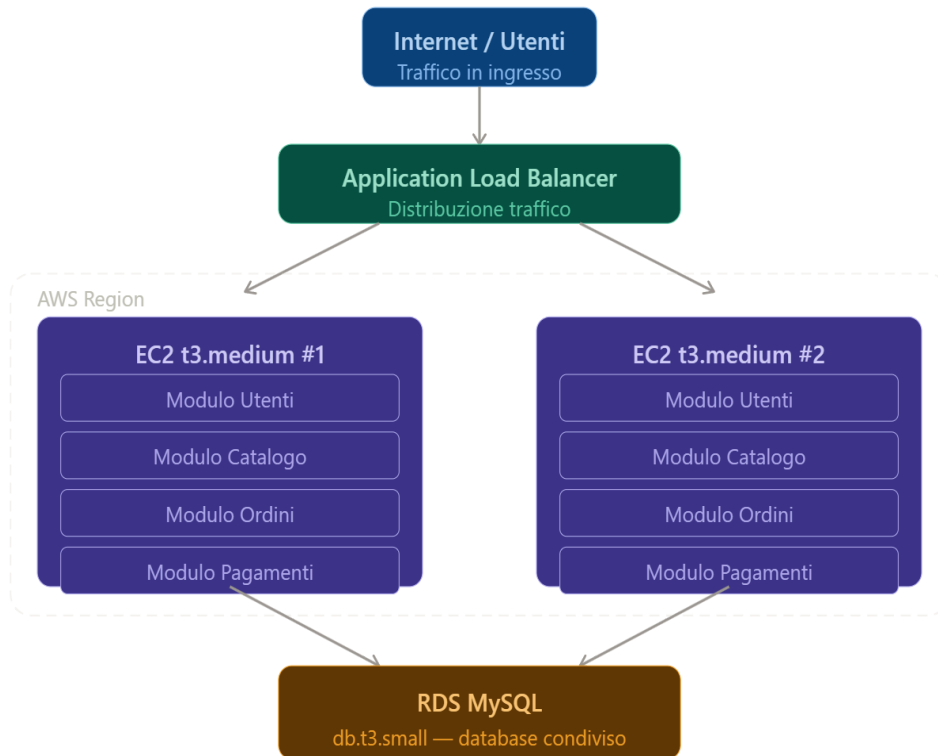
**Break-even serverless:** Immediato — non esiste un volume di traffico per cui EC2 always-on risulti più economico per questo carico di lavoro.

**Scalabilità Green:** All'aumentare del numero di pipeline (es. 10 processi CSV), il risparmio cresce linearmente:  $\$3.619/\text{anno}$  vs  $\$36.442/\text{anno}$  EC2.

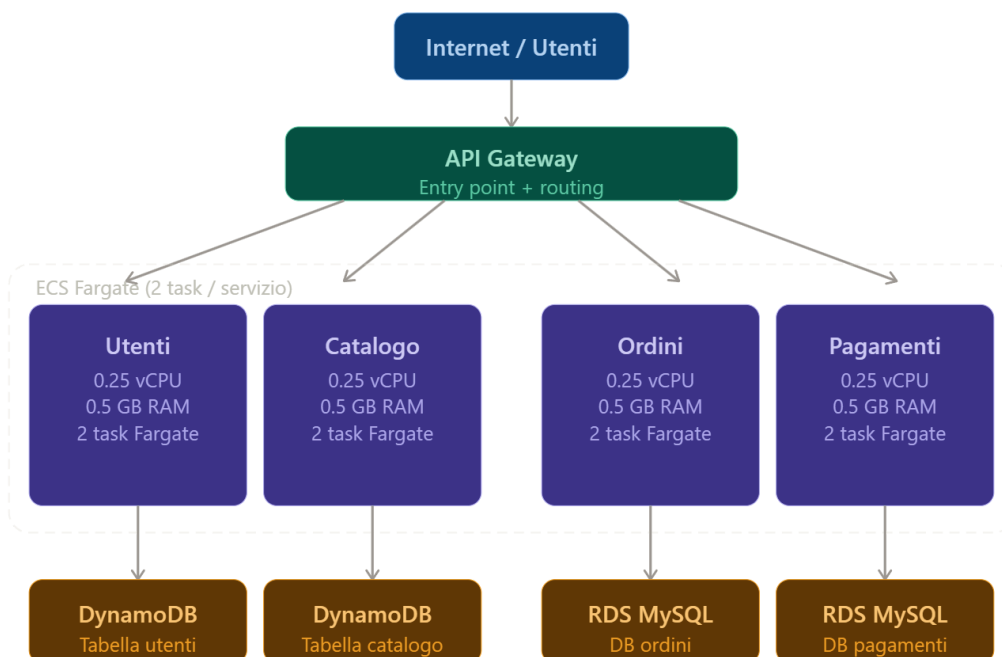
**Raccomandazione per EcoTech:** Adottare il paradigma serverless-first per tutti i nuovi workload event-driven, in linea con la strategia di Economia Circolare e dismissione responsabile degli asset IT (RAEE Step 1).

# Lab 03 - Fondamenti Serverless e Confronto Architetture

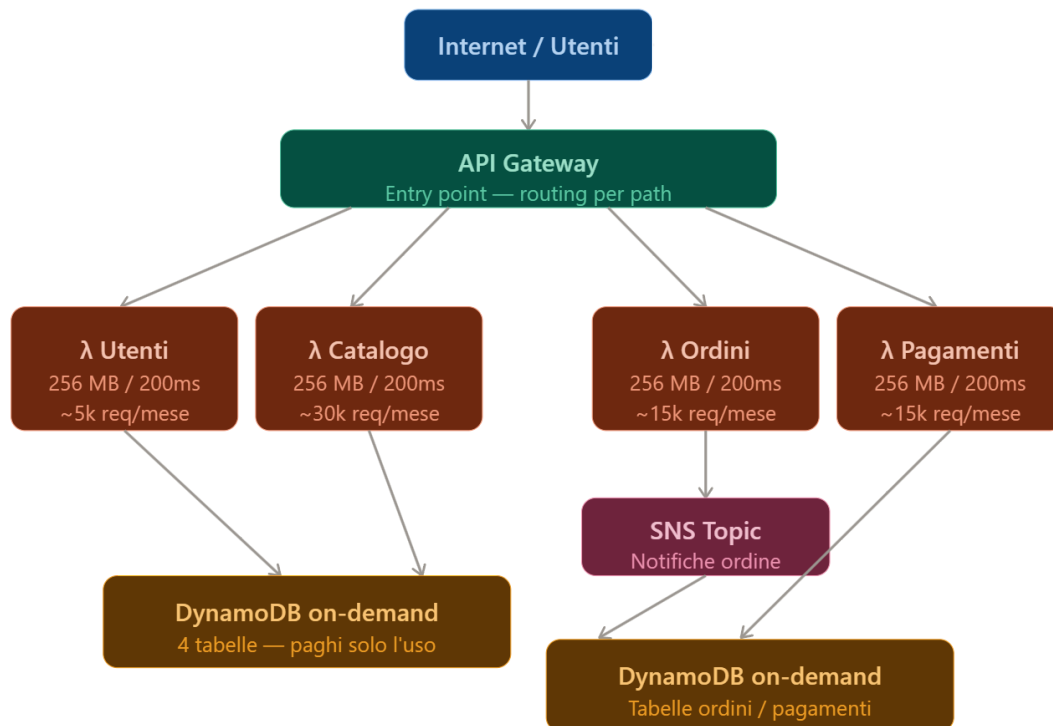
## ARCHITETTURA MONOLITICA



## ARCHITETTURA MICROSERVIZI



## ARCHITETTURA SERVERLESS



## TABELLA CONSUMI

| Monolitica | Microservizi | Serverless |
|------------|--------------|------------|
| ~181 \$    | ~124\$       | ~0\$       |

## TABELLA CONFRONTI

| Componente     | Monolite        | Microservizi       | Serverless       |
|----------------|-----------------|--------------------|------------------|
| Compute        | ~23 \$ (x2 EC2) | 0 \$ (x4 task Far) | 0 \$ (x4 Lambda) |
| Database       | ~135 \$         | ~31 \$ (x4 db)     | 0 \$ (x2 DB)     |
| API gateway    | //////////      | //////////         | ~0.02 \$         |
| Prezzo mensile | ~181 \$         | ~124 \$            | ~0 \$            |

## CONSIDERAZIONI FINALI

Raccomandazione: architettura serverless per QuickOrder

## **Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi**

Il traffico di QuickOrder è spiky: 500 ordini/giorno in media, ma fino a 3.000 ordini/ora nel weekend. Questo pattern di traffico è ideale per serverless:

- Costo variabile: si paga solo per le 15k richieste effettive/mese, non per server idle durante la notte.
- Scaling automatico: Lambda gestisce i picchi senza configurazione — da 0 a migliaia di esecuzioni in parallelo in millisecondi.
- Time-to-market: una startup paga ~\$0/mese invece di \$158 e si concentra sul prodotto, non sull'infra.

Limite da considerare: cold start Lambda (~200–500ms) per richieste rare. Mitigabile con Provisioned Concurrency se necessario.

[Provisioned Concurrency is an AWS Lambda feature that pre-initializes a specific number of execution environments, ensuring functions respond instantly by eliminating "[cold starts](#)"]

### **FONTI**

**monolite**

[AWS Pricing Calculator](#)

**microservizi**

[AWS Pricing Calculator](#)

**serverless**

[AWS Pricing Calculator](#)



# Lab 04 - Casi d'Uso Serverless e Analisi Economica

Regione utilizzata: us-east-1

## Step 1 - Scenario A: Blog con traffico scarso

### Architettura EC2

Costo anticipato

0,00 USD

Costo mensile

23,67 USD

Costo 12 months totale

284,04 USD

Include il costo anticipato

My Estimate

Duplica

Elimina

P

 Trova risorse

☐

Nome servizio

▼

Stato

▼

Costo anticipato

▼

Costo mensile

▼

☐

Amazon EC2



-

0,00 USD

8,23 USD

☐

Amazon RDS for Post...



-

0,00 USD

15,44 USD

<https://calculator.aws/#/estimate?id=5c3eb35a0ddf8cb345fd5b818054f0a6f2a23ee5>

### Architettura serverless

Costo anticipato

0,00 USD

Costo mensile

1,27 USD

Costo 12 months totale

15,24 USD

Include il costo anticipato

My Estimate

Duplica

Elimina

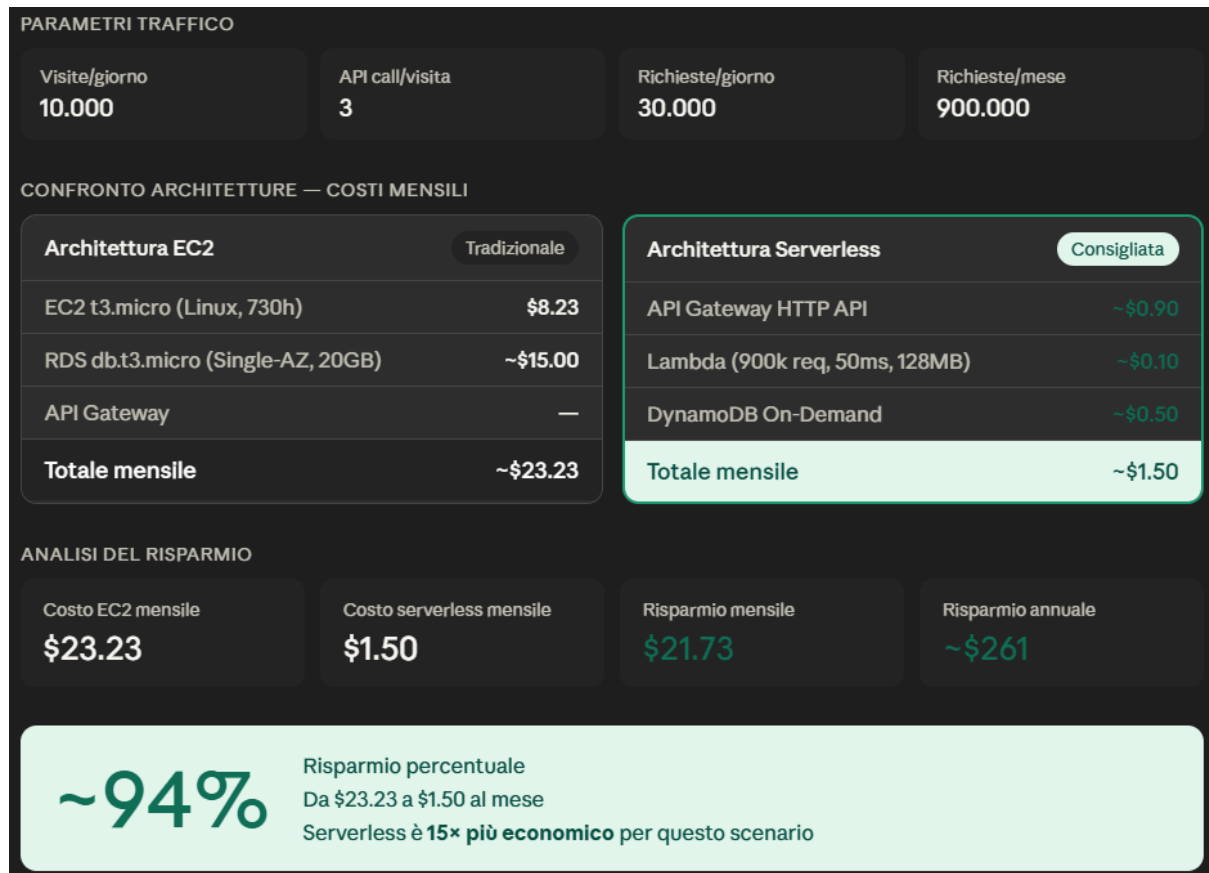
Pa

 Trova risorse

| <input type="checkbox"/> | Nome servizio      | Stato | Costo anticipato | Costo mensile |
|--------------------------|--------------------|-------|------------------|---------------|
| <input type="checkbox"/> | Amazon API Gateway | -     | 0,00 USD         | 0,90 USD      |
| <input type="checkbox"/> | AWS Lambda         | -     | 0,00 USD         | 0,00 USD      |
| <input type="checkbox"/> | Amazon DynamoDB    | -     | 0,00 USD         | 0,37 USD      |

<https://calculator.aws/#/estimate?id=0d9e6a88a410e0ee61d104cb27945312edd69e6e>

## Conclusioni



Per un blog a basso traffico, la scelta serverless è quella giusta. Il costo fisso dell'architettura EC2 (\$23.23/mese) viene pagato indipendentemente dal numero di visite: che il blog riceva 10.000 visitatori o zero, la spesa rimane invariata. Con l'architettura serverless invece si paga esclusivamente per ciò che si consuma, portando il costo mensile a soli \$1.50.

Il risparmio del 94% — circa \$261 all'anno — rende il serverless la scelta economicamente superiore per questo tipo di carico di lavoro leggero e variabile. La maggiore complessità di configurazione (Lambda, API Gateway, DynamoDB) è ampiamente ripagata già nel primo mese di esercizio.

## Step 2 - Scenario B: E-commerce ad alto traffico costante

### Architettura EC2

Costo anticipato  
0,00 USD

Costo mensile  
649,17 USD

Costo 12 months totale  
**7790,04 USD**  
Include il costo anticipato

| My Estimate                                |                        |  |       |  |                  | Duplica | Elimina       | Passa |
|--------------------------------------------|------------------------|--|-------|--|------------------|---------|---------------|-------|
| <input type="text" value="Trova risorse"/> |                        |  |       |  |                  |         |               |       |
| <input type="checkbox"/>                   | Nome servizio          |  | Stato |  | Costo anticipato |         | Costo mensile |       |
| <input type="checkbox"/>                   | Amazon EC2             |  | -     |  | 0,00 USD         |         | 626,90 USD    |       |
| <input type="checkbox"/>                   | Elastic Load Balancing |  | -     |  | 0,00 USD         |         | 22,27 USD     |       |

<https://calculator.aws/#/estimate?id=fb6f0b20606a20da89d89de9d71705346279f6d4>

### Architettura serverless

Costo anticipato  
0,00 USD

Costo mensile  
2015,14 USD

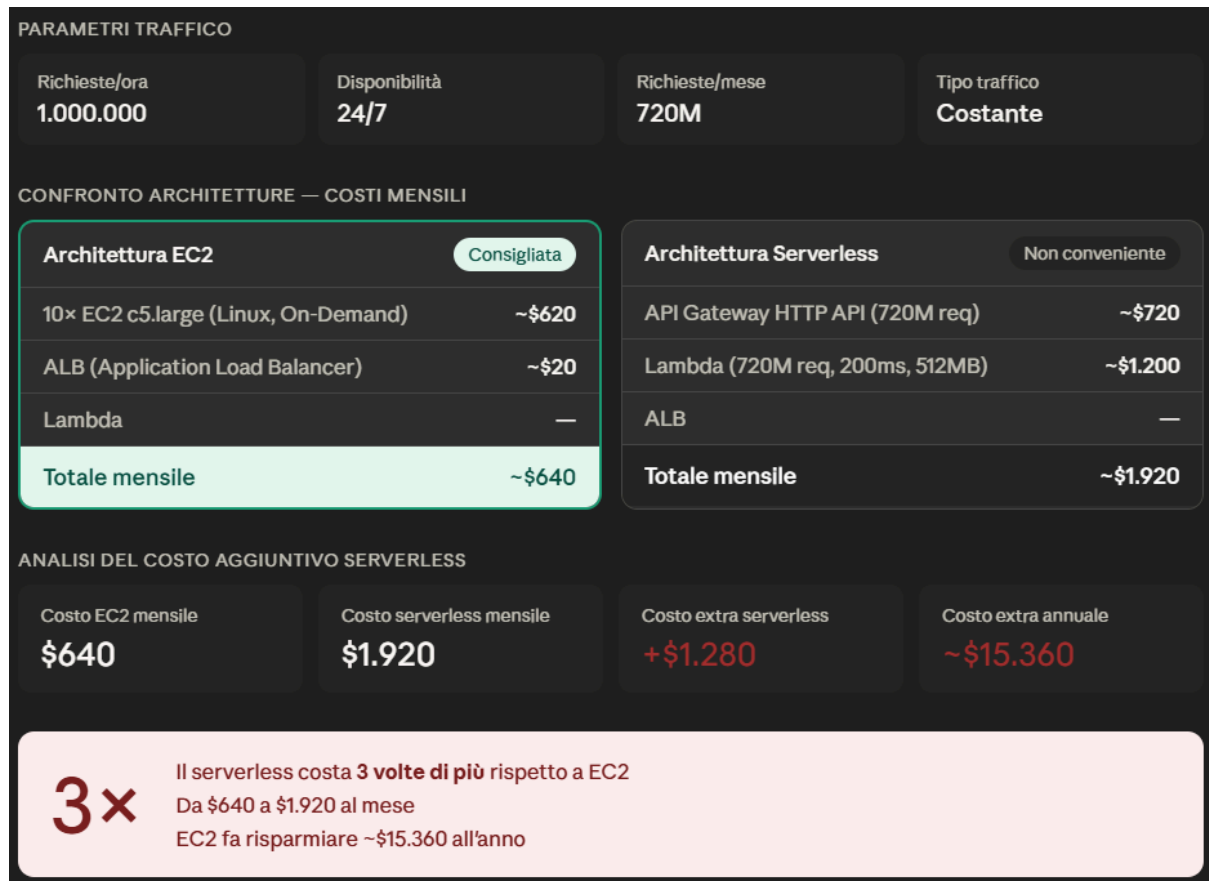
Costo 12 months totale  
**24.181,68 USD**  
Include il costo anticipato

| My Estimate                                |  |  |  |  |  | Duplica | Elimina | Passa |
|--------------------------------------------|--|--|--|--|--|---------|---------|-------|
| <input type="text" value="Trova risorse"/> |  |  |  |  |  |         |         |       |

| My Estimate                                |                    |  |       |  |                  | Duplica | Elimina       | Passa |
|--------------------------------------------|--------------------|--|-------|--|------------------|---------|---------------|-------|
| <input type="text" value="Trova risorse"/> |                    |  |       |  |                  |         |               |       |
| <input type="checkbox"/>                   | Nome servizio      |  | Stato |  | Costo anticipato |         | Costo mensile |       |
| <input type="checkbox"/>                   | Amazon API Gateway |  | -     |  | 0,00 USD         |         | 678,00 USD    |       |
| <input type="checkbox"/>                   | AWS Lambda         |  | -     |  | 0,00 USD         |         | 1337,14 USD   |       |

<https://calculator.aws/#/estimate?id=432cdffbba8e0339a65d9f58fd3a9a50d461b5fd>

## Conclusioni



In questo scenario la situazione si ribalta completamente rispetto al blog. Un e-commerce con 1 milione di richieste API all'ora, attivo 24 ore su 24 e 7 giorni su 7, genera 720 milioni di richieste mensili — un volume che il modello serverless non riesce a sostenere economicamente. API Gateway e Lambda fatturano ogni singola invocazione, portando il costo a circa \$1.920 al mese, quasi tre volte il costo del cluster EC2 da \$640.

Il traffico costante è il fattore chiave: con EC2 si paga un canone fisso indipendentemente dal numero di richieste, il che diventa un vantaggio enorme quando il carico è elevato e prevedibile. Scegliere serverless in questo contesto significherebbe spendere circa \$15.360 in più ogni anno senza alcun beneficio operativo.

### Step 3 - Scenario C: Applicazione stagionale

#### **Architettura EC2 estiva**

Costo anticipato  
0,00 USD

Costo mensile  
270,47 USD

Costo 12 months totale  
**3245,64 USD**  
Include il costo anticipato

| My Estimate                                |                        |  |       |  |                  | Duplica | Elimina       | Pe |
|--------------------------------------------|------------------------|--|-------|--|------------------|---------|---------------|----|
| <input type="text" value="Trova risorse"/> |                        |  |       |  |                  |         |               |    |
| <input type="checkbox"/>                   | Nome servizio          |  | Stato |  | Costo anticipato |         | Costo mensile |    |
| <input type="checkbox"/>                   | Amazon EC2             |  | -     |  | 0,00 USD         |         | 248,20 USD    |    |
| <input type="checkbox"/>                   | Elastic Load Balancing |  | -     |  | 0,00 USD         |         | 22,27 USD     |    |

<https://calculator.aws/#/estimate?id=c20e013e63d4da4a3e6531a3a33ee8fb7b589dd6>

#### **Architettura EC2 invernale**

In inverno con EC2 il concetto non cambia, l'unica differenza è che si moltiplica il risultato mensile per 8 mesi invece di 4

Costo anticipato  
0,00 USD

Costo mensile  
270,47 USD

Costo 12 months totale  
**3245,64 USD**  
Include il costo anticipato

| My Estimate                                |                        |  |       |  |                  | Duplica | Elimina       | Pe |
|--------------------------------------------|------------------------|--|-------|--|------------------|---------|---------------|----|
| <input type="text" value="Trova risorse"/> |                        |  |       |  |                  |         |               |    |
| <input type="checkbox"/>                   | Nome servizio          |  | Stato |  | Costo anticipato |         | Costo mensile |    |
| <input type="checkbox"/>                   | Amazon EC2             |  | -     |  | 0,00 USD         |         | 248,20 USD    |    |
| <input type="checkbox"/>                   | Elastic Load Balancing |  | -     |  | 0,00 USD         |         | 22,27 USD     |    |

## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

### Architettura serverless estiva

Costo anticipato  
0,00 USD

Costo mensile  
96,33 USD

Costo 12 months totale  
**1155,96 USD**  
Include il costo anticipato

| My Estimate                                |                    |                                                                                   |       |  | Duplica          | Elimina       | Pa |
|--------------------------------------------|--------------------|-----------------------------------------------------------------------------------|-------|--|------------------|---------------|----|
| <input type="text" value="Trova risorse"/> |                    |                                                                                   |       |  |                  |               |    |
| <input type="checkbox"/>                   | Nome servizio      |                                                                                   | Stato |  | Costo anticipato | Costo mensile |    |
| <input type="checkbox"/>                   | Amazon API Gateway |  | -     |  | 0,00 USD         | 36,00 USD     |    |
| <input type="checkbox"/>                   | AWS Lambda         |  | -     |  | 0,00 USD         | 60,33 USD     |    |

<https://calculator.aws/#/estimate?id=0658bc37b2aef3ed6dea9e5bf75e0748a0b33211>

### Architettura serverless invernale

Costo anticipato  
0,00 USD

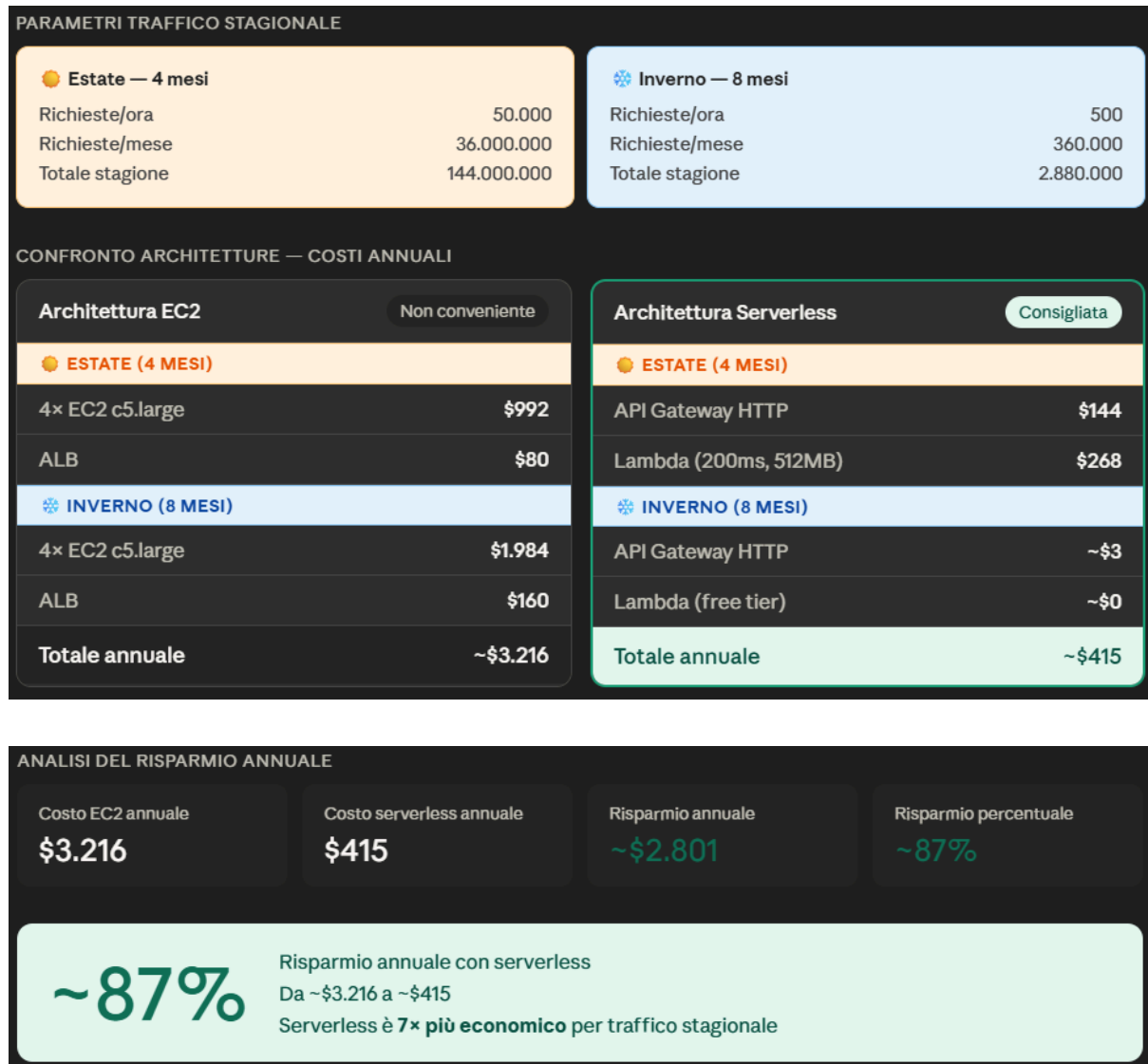
Costo mensile  
0,36 USD

Costo 12 months totale  
**4,32 USD**  
Include il costo anticipato

| My Estimate                                |                    |                                                                                     |       |  | Duplica          | Elimina       | Pa |
|--------------------------------------------|--------------------|-------------------------------------------------------------------------------------|-------|--|------------------|---------------|----|
| <input type="text" value="Trova risorse"/> |                    |                                                                                     |       |  |                  |               |    |
| <input type="checkbox"/>                   | Nome servizio      |                                                                                     | Stato |  | Costo anticipato | Costo mensile |    |
| <input type="checkbox"/>                   | Amazon API Gateway |  | -     |  | 0,00 USD         | 0,36 USD      |    |
| <input type="checkbox"/>                   | AWS Lambda         |  | -     |  | 0,00 USD         | 0,00 USD      |    |

<https://calculator.aws/#/estimate?id=ef81b68e7fda0b574cc509291b1016a36d195b85>

## Conclusioni



Il traffico stagionale è il contesto in cui il serverless esprime il suo massimo vantaggio. Un'applicazione di prenotazioni vacanze non può permettersi di sottodimensionare il cluster estivo, ma con EC2 quella capacità va pagata tutto l'anno: i \$268/mese del cluster vengono addebitati anche in gennaio, quando il servizio riceve appena 500 richieste all'ora.

Con l'architettura serverless invece i costi seguono il traffico in modo preciso: in estate Lambda scala automaticamente per gestire i 50.000 req/ora, in inverno il volume è così basso da rientrare nel free tier permanente di AWS, portando il costo mensile quasi a zero. Il risparmio annuale di circa \$2.801 — pari all'87% — rende il serverless la scelta nettamente superiore per qualsiasi applicazione con pattern di utilizzo irregolare o stagionale.

## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

\*per ottenere i risultati delle tabelle bisogna moltiplicare X4 (estate), X8 (inverno)

### **Break-Even Point**

- con Lambda da 200ms e 256 MB ( $256\text{MB} * 0.2\text{s} = 51.2 \text{ MB-second}$ ) il costo per ogni invocazione sopra 1M in eu-south-1 è:

$\$0.00000023$  (requests)  
+  $\$0.0000195172$  per ogni GB-second sopra i 400.000  
GB-second (compute)

- supponiamo di avere 19M di richieste/mese (18M, perché uno è di free tier):

$(0.23\$ * 18 \rightarrow \text{requests}) + \sim 10\$ \rightarrow \text{compute} = \sim 14.14\$/\text{mese}$

- aggiungiamo 19M/mese di http API requests a 1\$/1M requests:

$14.14\$/\text{mese} + 19\$/\text{mese} = 33.14\$/\text{mese}$

- con EC2 t3.medium:  $\sim 33\$/\text{mese}$

↳ il break-even point tra una Lambda da 200ms / 256 MB e una EC2 t3.medium 24/7 è di circa **19M di richieste Lambda al mese**



## Lab 05 - Limitazioni Serverless e Best Practice

### Scenario

Un collega ha scritto una Lambda function per elaborare ordini e-commerce. Il codice funziona ma viola diverse best practice. Devi fare una code review e proporre miglioramenti.

### Step 1 - Analizza il codice

```
import boto3
import json
import os
import pandas # unused
import numpy # unused

DB_PASSWORD = "SuperSecret123!" # hardcoded credential

def lambda_handler(event, context):
    # Initialize DynamoDB client inside handler
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table('orders')

    # No input validation
    order_id = event['body']['order_id']
    amount = event['body']['amount']

    # Write to DynamoDB
    table.put_item(Item={
        'order_id': order_id,
        'amount': amount,
        'db_password': DB_PASSWORD # storing password in DB!
    })

    # Connect to RDS with hardcoded password
    import pymysql
    conn = pymysql.connect(
        host='mydb.cluster-xxx.us-east-1.rds.amazonaws.com',
        user='admin',
        password=DB_PASSWORD,
        database='orders'
    )

    # No error handling
    cursor = conn.cursor()
    cursor.execute(f"INSERT INTO orders VALUES ({order_id}, {amount})")
    conn.commit()
    conn.close()

    return {'statusCode': 200, 'body': 'OK'}
```

**Step 2 - Identifica i problemi**

| Problema                                                                                                                                      | Soluzione | Categoria   | Livello di Rischio |
|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------|--------------------|
| <b>Credenziali hardcoded</b><br><br>DB_PASSWORD scritto nel sorgente: visibile a chiunque acceda al repo                                      |           | Sicurezza   | <b>Alto</b>        |
| <b>Password salvata in DynamoDB</b><br><br>db_password scritto come attributo dell'item: credenziale esposta nel DB                           |           | Sicurezza   | <b>Alto</b>        |
| <b>Librerie inutilizzate (pandas, numpy)</b><br><br>Aumentano le dimensioni del package e il cold-start time senza utilità                    |           | Performance | <b>Basso</b>       |
| <b>Init e conn del DB eseguita dentro l'handler</b><br><br>Vengono eseguite ad ogni chiamata. Se le mettessimo fuori verrebbero chiamate solo |           |             |                    |

**Gruppo 2:** *Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi*

|                                                                                                                     |                                                |  |   |
|---------------------------------------------------------------------------------------------------------------------|------------------------------------------------|--|---|
| alla creazione del container e poi mantenute per 5-15 min                                                           |                                                |  |   |
| <b>No error fallback nell'init del DB</b>                                                                           |                                                |  |   |
| <b>No input validation</b><br><br>Espone ad errori<br>Insieme alla query inserita in chiaro, espone a SQL injection |                                                |  |   |
| <b>Password del DB salvata in chiaro nel DB</b>                                                                     |                                                |  |   |
| <b>No fallback della conn al DB</b>                                                                                 |                                                |  |   |
| <b>Import di pymysql a metà del codice</b>                                                                          | <b>Scriverlo in cima al file per chiarezza</b> |  | Q |
| <b>Query SQL in chiaro</b><br><br>Espone a SQL injection                                                            |                                                |  |   |
| <b>no error handling della query MySQL</b>                                                                          | <b>try: ... except<br/>MySQLdb.Error:</b>      |  |   |
| <b>Return non tiene conto di errori</b>                                                                             |                                                |  |   |

**Gruppo 2:** *Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi*

|                                      |               |  |  |
|--------------------------------------|---------------|--|--|
| No salvataggio di logs su CloudWatch | import logger |  |  |
|--------------------------------------|---------------|--|--|

## Lab 06 - Prima Funzione Lambda

Matteo Ninotti:

### 128mb

Ho notato che “Billed duration” dell’execution *cold start* è esattamente = “Init duration” + “Duration” (92 ms)

Invece “Billed duration” dell’execution *warm start* è semplicemente “Duration”

**cold:** Duration: 2.02 ms   Billed Duration: 92 ms   Memory Size: 128 MB   Max Memory Used: 37 MB   Init Duration: 89.77 ms

**warm:** Duration: 1.65 ms   Billed Duration: 2 ms   Memory Size: 256 MB   Max Memory Used: 37 MB

**cold:** Duration: 2.01 ms   Billed Duration: 88 ms   Memory Size: 128 MB   Max Memory Used: 37 MB   Init Duration: 85.89 ms

**warm:** Duration: 1.86 ms   Billed Duration: 2 ms   Memory Size: 128 MB   Max Memory Used: 37 MB

The screenshot shows the AWS Lambda console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and account information (United States (N. Virginia), Account ID: 0648-1451-6769). Below the navigation bar, the breadcrumb trail indicates the path: CloudWatch > Log management > /aws/lambda/hello-serverless > 2026/05/07/[[\$LATEST]]09d370075c724c59a2331ed3544dfa74. The main section is titled 'Log events' and includes a search bar and filters. The log events are displayed in a table with columns for Timestamp and Message. The events show the function being invoked, the request being processed, and the response being returned. The log events are as follows:

| Timestamp                | Message                                                                                                                                                                       |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2026-05-07T13:38:24.284Z | INIT_START Runtime Version: python:3.12.mainlinev2.v7 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:e4ab553846c4e081013ff7d1d608a5358d5b956bb5b81c83c66d2a31da08f624 |
| 2026-05-07T13:38:24.378Z | START RequestId: 40b3dab7-c820-4858-ad5a-574bdb720fc7 Version: \$LATEST                                                                                                       |
| 2026-05-07T13:38:24.378Z | Processed request for: Mario                                                                                                                                                  |
| 2026-05-07T13:38:24.378Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "128", "remaining_time_ms": 3000, "execution_time_ms": 0.01}                   |
| 2026-05-07T13:38:24.381Z | END RequestId: 40b3dab7-c820-4858-ad5a-574bdb720fc7                                                                                                                           |
| 2026-05-07T13:38:24.381Z | REPORT RequestId: 40b3dab7-c820-4858-ad5a-574bdb720fc7 Duration: 2.02 ms Billed Duration: 92 ms Memory Size: 128 MB Max Memory Used: 37 MB Init Duration: 89.77 ms            |
| 2026-05-07T13:38:37.990Z | START RequestId: ac8e6c2a-c7fb-47f2-b58b-9bcfe1a105e3 Version: \$LATEST                                                                                                       |
| 2026-05-07T13:38:37.990Z | START RequestId: ac8e6c2a-c7fb-47f2-b58b-9bcfe1a105e3 Version: \$LATEST                                                                                                       |
| 2026-05-07T13:38:37.990Z | Processed request for: Mario                                                                                                                                                  |
| 2026-05-07T13:38:37.990Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "128", "remaining_time_ms": 3000, "execution_time_ms": 0.01}                   |
| 2026-05-07T13:38:37.999Z | END RequestId: ac8e6c2a-c7fb-47f2-b58b-9bcfe1a105e3                                                                                                                           |
| 2026-05-07T13:38:37.999Z | REPORT RequestId: ac8e6c2a-c7fb-47f2-b58b-9bcfe1a105e3 Duration: 1.61 ms Billed Duration: 2 ms Memory Size: 128 MB Max Memory Used: 37 MB                                     |

### 256mb

...Nel mio caso, aumentando la memoria a 256MB, entrambe “Init duration” e “Duration” sono durate più tempo

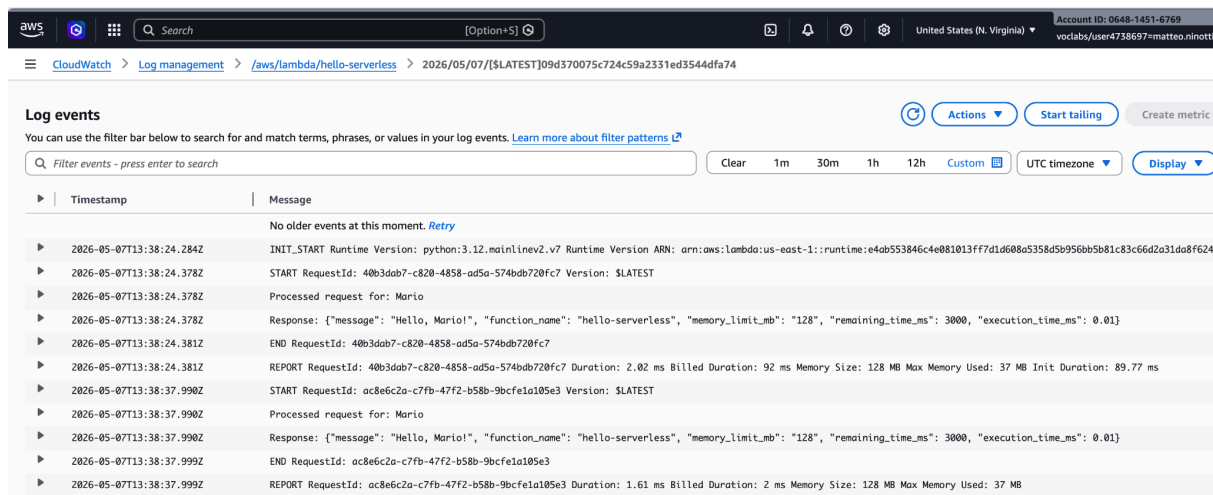
**cold:** Duration: 2.17 ms   Billed Duration: 97 ms   Memory Size: 256 MB   Max Memory Used: 36 MB   Init Duration: 94.71 ms

**warm:** Duration: 1.88 ms   Billed Duration: 2 ms   Memory Size: 256 MB   Max Memory Used: 36 MB

**warm:** Duration: 1.65 ms   Billed Duration: 2 ms   Memory Size: 256 MB   Max Memory Used: 37 MB

**warm:** Duration: 1.72 ms   Billed Duration: 2 ms   Memory Size: 256 MB   Max Memory Used: 37 MB

## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi



The screenshot shows the AWS CloudWatch console interface for a Lambda function named 'hello-serverless'. The 'Log events' tab is selected, displaying a list of log events. The events include initialization, request processing, and response generation. Key metrics like 'Billed Duration' and 'Init Duration' are visible in the log messages.

| Timestamp                | Message                                                                                                                                                                     |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2026-05-07T13:38:24.284Z | INIT_START Runtime Version: python:3.12.mainlinev2.v7 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:e4ab553846c4e081013ff7d1d608a5358d5b956b5b81c83c66d2a31da8f624 |
| 2026-05-07T13:38:24.378Z | START RequestId: 40b3dab7-c820-4858-ad5a-574bdb720fc7 Version: \$LATEST                                                                                                     |
| 2026-05-07T13:38:24.378Z | Processed request for: Mario                                                                                                                                                |
| 2026-05-07T13:38:24.378Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "128", "remaining_time_ms": 3000, "execution_time_ms": 0.01}                 |
| 2026-05-07T13:38:24.381Z | END RequestId: 40b3dab7-c820-4858-ad5a-574bdb720fc7                                                                                                                         |
| 2026-05-07T13:38:24.381Z | REPORT RequestId: 40b3dab7-c820-4858-ad5a-574bdb720fc7 Duration: 2.02 ms Billed Duration: 92 ms Memory Size: 128 MB Max Memory Used: 37 MB Init Duration: 89.77 ms          |
| 2026-05-07T13:38:37.990Z | START RequestId: ac8e6c2a-c7fb-47f2-b58b-9bcfe1a105e3 Version: \$LATEST                                                                                                     |
| 2026-05-07T13:38:37.990Z | Processed request for: Mario                                                                                                                                                |
| 2026-05-07T13:38:37.990Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "128", "remaining_time_ms": 3000, "execution_time_ms": 0.01}                 |
| 2026-05-07T13:38:37.999Z | END RequestId: ac8e6c2a-c7fb-47f2-b58b-9bcfe1a105e3                                                                                                                         |
| 2026-05-07T13:38:37.999Z | REPORT RequestId: ac8e6c2a-c7fb-47f2-b58b-9bcfe1a105e3 Duration: 1.61 ms Billed Duration: 2 ms Memory Size: 128 MB Max Memory Used: 37 MB                                   |

La tabella seguente compara i valori di *Billed Duration* e *Init Duration* per le dimensioni di memoria specificate, basandosi sui dati di **cold start** trovati nelle note del Lab 06:

| Dimensione Memoria | Billed Duration (ms) | Init Duration (ms) |
|--------------------|----------------------|--------------------|
| 128 MB             | 92                   | 89.77              |
| 256 MB             | 97                   | 94.71              |

### 1024mb

... anche con 1024mb di memoria, i tempi non cambiano significativamente:

**cold:** Duration: 2.04 ms   Billed Duration: 92 ms   Memory Size: 1024 MB   Max Memory Used: 36 MB   Init Duration: 89.88 ms

### conclusione:

confrontiamo i valori di “Max Memory Used”: indipendentemente dal taglio di memoria, sono sempre attorno a 36-37mb.

Probabilmente la task della lambda (ritornare json “hello + \$nome”) è troppo leggera per beneficiare di maggiore memoria.

**risultato:** i tempi di execution restano **pressoché costanti**

## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

**Riccardo Gouthier:**

### **Lambda 128:**

Ho notato che “Billed duration” dell’execution *cold start* è esattamente = “Init duration” + “Duration” (93 ms)

**cold:** REPORT RequestId: 6d3b7e6c-c3a1-435a-9bd1-b721f20025a0 Duration: 4.27 ms  
Billed Duration: 93 ms Memory Size: 128 MB  
Max Memory Used: 36 MB Init Duration: 88.10 ms

**warm:** REPORT RequestId: 75bc7cf5-075a-4107-96a9-3541eda00a63 Duration: 13.61 ms  
Billed Duration: 14 ms Memory Size: 128 MB  
Max Memory Used: 36 MB

|   |                          |                                                                                                                                                                           |
|---|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ▶ | 2026-05-07T13:37:16.984Z | Processed request for: Mario                                                                                                                                              |
| ▶ | 2026-05-07T13:37:16.984Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "128", "remaining_time_ms": 3000, "execution_time_ms": 0.01}               |
| ▶ | 2026-05-07T13:37:16.989Z | END RequestId: 6d3b7e6c-c3a1-435a-9bd1-b721f20025a0                                                                                                                       |
| ▼ | 2026-05-07T13:37:16.989Z | <b>REPORT</b> RequestId: 6d3b7e6c-c3a1-435a-9bd1-b721f20025a0 Duration: 4.27 ms Billed Duration: 93 ms Memory Size: 128 MB Max Memory Used: 36 MB Init Duration: 88.10 ms |
|   |                          | <b>REPORT</b> RequestId: 6d3b7e6c-c3a1-435a-9bd1-b721f20025a0 Duration: 4.27 ms Billed Duration: 93 ms Memory Size: 128 MB Max Memory Used: 36 MB Init Duration: 88.10 ms |
| ▶ | 2026-05-07T13:39:28.694Z | START RequestId: 75bc7cf5-075a-4107-96a9-3541eda00a63 Version: \$LATEST                                                                                                   |
| ▼ | 2026-05-07T13:39:28.694Z | Processed request for: Mario                                                                                                                                              |
|   |                          | Processed request for: Mario                                                                                                                                              |
| ▶ | 2026-05-07T13:39:28.694Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "128", "remaining_time_ms": 2999, "execution_time_ms": 0.01}               |
| ▶ | 2026-05-07T13:39:28.708Z | END RequestId: 75bc7cf5-075a-4107-96a9-3541eda00a63                                                                                                                       |
| ▼ | 2026-05-07T13:39:28.708Z | <b>REPORT</b> RequestId: 75bc7cf5-075a-4107-96a9-3541eda00a63 Duration: 13.61 ms Billed Duration: 14 ms Memory Size: 128 MB Max Memory Used: 36 MB                        |
|   |                          | <b>REPORT</b> RequestId: 75bc7cf5-075a-4107-96a9-3541eda00a63 Duration: 13.61 ms Billed Duration: 14 ms Memory Size: 128 MB Max Memory Used: 36 MB                        |
| ▶ | 2026-05-07T13:39:29.591Z | START RequestId: 122fe363-73bf-47f5-9642-d9d9eed5a8d2 Version: \$LATEST                                                                                                   |
| ▶ | 2026-05-07T13:39:29.592Z | Processed request for: Mario                                                                                                                                              |

### **Lambda 256:**

Ho notato che “Billed duration” dell’execution *cold start* è esattamente = “Init duration” + “Duration” (103 ms)

**cold:** REPORT RequestId: bd298115-2fe2-442e-abc1-f4c3d457e703 Duration: 2.23 ms  
Billed Duration: 103 ms Memory Size: 256 MB  
Max Memory Used: 36 MB Init Duration: 99.93 ms

**warm:** REPORT RequestId: a7ea219a-2f00-45ff-b052-2baf017d9b29 Duration: 1.82 ms  
Billed Duration: 2 ms  
Memory Size: 256 MB  
Max Memory Used: 36 MB

**Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi**

No older events at this moment. [Retry](#)

|   |                          |                                                                                                                                                                                                                                                                                                                                                       |
|---|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ▶ | 2026-05-07T13:57:53.545Z | INIT_START Runtime Version: python:3.12.mainlinev2.v7 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:e4ab553846c4e081013ff7d1d608a5358d5b956bb5b81c83c66d2a31da8f6244                                                                                                                                                                         |
| ▶ | 2026-05-07T13:57:53.648Z | START RequestId: bd298115-2fe2-442e-abc1-f4c3d457e703 Version: \$LATEST                                                                                                                                                                                                                                                                               |
| ▶ | 2026-05-07T13:57:53.649Z | Processed request for: Mario                                                                                                                                                                                                                                                                                                                          |
| ▶ | 2026-05-07T13:57:53.649Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "256", "remaining_time_ms": 2999, "execution_time_ms": 0.01}                                                                                                                                                                                           |
| ▶ | 2026-05-07T13:57:53.652Z | END RequestId: bd298115-2fe2-442e-abc1-f4c3d457e703                                                                                                                                                                                                                                                                                                   |
| ▼ | 2026-05-07T13:57:53.652Z | <b>REPORT</b> RequestId: bd298115-2fe2-442e-abc1-f4c3d457e703 Duration: 2.23 ms Billed Duration: 103 ms Memory Size: 256 MB Max Memory Used: 36 MB Init Duration: 99.93 ms<br><br>REPORT RequestId: bd298115-2fe2-442e-abc1-f4c3d457e703 Duration: 2.23 ms Billed Duration: 103 ms Memory Size: 256 MB Max Memory Used: 36 MB Init Duration: 99.93 ms |
| ▶ | 2026-05-07T13:57:55.008Z | START RequestId: a7ea219a-2f00-45ff-b052-2baef017d9b29 Version: \$LATEST                                                                                                                                                                                                                                                                              |
| ▶ | 2026-05-07T13:57:55.008Z | Processed request for: Mario                                                                                                                                                                                                                                                                                                                          |
| ▶ | 2026-05-07T13:57:55.008Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "256", "remaining_time_ms": 3000, "execution_time_ms": 0.01}                                                                                                                                                                                           |
| ▶ | 2026-05-07T13:57:55.010Z | END RequestId: a7ea219a-2f00-45ff-b052-2baef017d9b29                                                                                                                                                                                                                                                                                                  |
| ▼ | 2026-05-07T13:57:55.010Z | <b>REPORT</b> RequestId: a7ea219a-2f00-45ff-b052-2baef017d9b29 Duration: 1.82 ms Billed Duration: 2 ms Memory Size: 256 MB Max Memory Used: 36 MB<br><br>REPORT RequestId: a7ea219a-2f00-45ff-b052-2baef017d9b29 Duration: 1.82 ms Billed Duration: 2 ms Memory Size: 256 MB Max Memory Used: 36 MB                                                   |

La tabella seguente compara i valori di *Billed Duration* e *Init Duration* per le dimensioni di memoria specificate, basandosi sui dati di **cold start** trovati nelle note del Lab 06:

| Dimensione Memoria | Billed Duration (ms) | Init Duration (ms) |
|--------------------|----------------------|--------------------|
| 128 MB             | 93                   | 88.10              |
| 256 MB             | 103                  | 99.93              |



Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

Ivan Cioffi:

Lambda 128MB

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

|  | Timestamp                | Message                                                                                                                                                                       |
|--|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |                          | No older events at this moment. <a href="#">Retry</a>                                                                                                                         |
|  | 2026-05-11T09:27:42.378Z | INIT_START Runtime Version: python:3.12.mainlinev2.v7 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:e4ab553846c4e081013ff7d1d608a5358d5b956bb5b81c83c66d2a31da8f6244 |
|  | 2026-05-11T09:27:42.463Z | START RequestId: c3d39a77-ddeb-4927-a432-c09dad18cbf0 Version: \$LATEST                                                                                                       |
|  | 2026-05-11T09:27:42.463Z | Processed request for: Mario                                                                                                                                                  |
|  | 2026-05-11T09:27:42.463Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "128", "remaining_time_ms": 2999, "execution_time_ms": 0.01}                   |
|  | 2026-05-11T09:27:42.474Z | END RequestId: c3d39a77-ddeb-4927-a432-c09dad18cbf0                                                                                                                           |
|  | 2026-05-11T09:27:42.474Z | REPORT RequestId: c3d39a77-ddeb-4927-a432-c09dad18cbf0 Duration: 1.85 ms Billed Duration: 83 ms Memory Size: 128 MB Max Memory Used: 36 MB Init Duration: 80.88 ms            |
|  | 2026-05-11T09:29:16.879Z | START RequestId: 5e634a07-7baf-4840-aacb-49b7793bb89f Version: \$LATEST                                                                                                       |
|  | 2026-05-11T09:29:16.880Z | Processed request for: Mario                                                                                                                                                  |
|  | 2026-05-11T09:29:16.880Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "128", "remaining_time_ms": 2999, "execution_time_ms": 0.01}                   |
|  | 2026-05-11T09:29:16.896Z | END RequestId: 5e634a07-7baf-4840-aacb-49b7793bb89f                                                                                                                           |
|  | 2026-05-11T09:29:16.896Z | REPORT RequestId: 5e634a07-7baf-4840-aacb-49b7793bb89f Duration: 15.09 ms Billed Duration: 16 ms Memory Size: 128 MB Max Memory Used: 36 MB                                   |
|  |                          | No newer events at this moment. <i>Auto retry paused.</i> <a href="#">Resume</a>                                                                                              |

Lambda 256MB

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

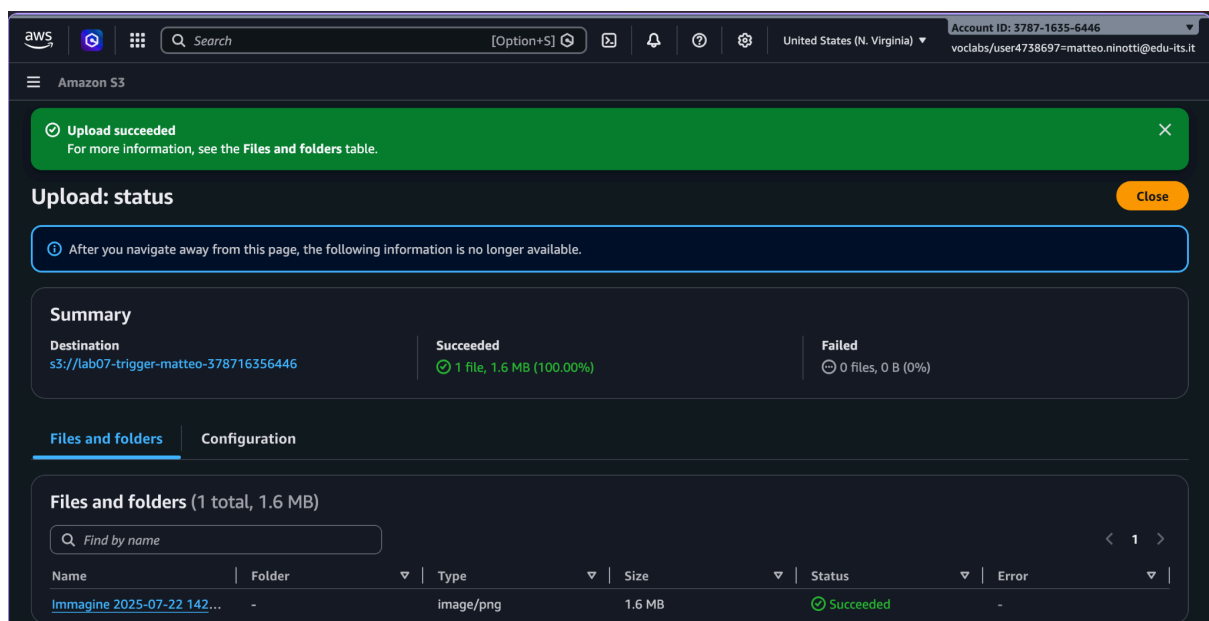
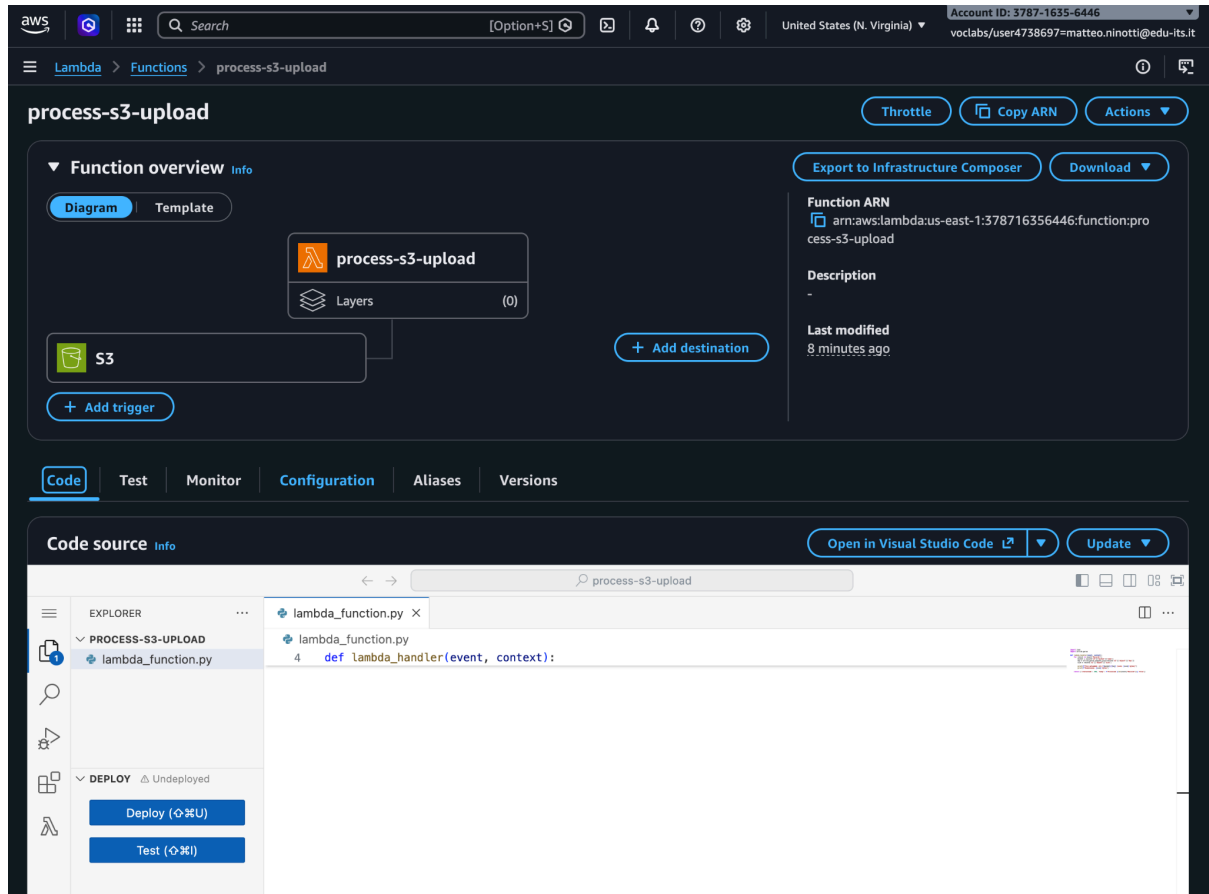
Filter events - press enter to search

|  | Timestamp                | Message                                                                                                                                                                       |
|--|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |                          | No older events at this moment. <a href="#">Retry</a>                                                                                                                         |
|  | 2026-05-11T09:56:02.802Z | INIT_START Runtime Version: python:3.12.mainlinev2.v7 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:e4ab553846c4e081013ff7d1d608a5358d5b956bb5b81c83c66d2a31da8f6244 |
|  | 2026-05-11T09:56:02.891Z | START RequestId: 006d7ac6-d897-4b39-903e-f568e02ce16a Version: \$LATEST                                                                                                       |
|  | 2026-05-11T09:56:02.892Z | Processed request for: Mario                                                                                                                                                  |
|  | 2026-05-11T09:56:02.892Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "256", "remaining_time_ms": 2999, "execution_time_ms": 0.01}                   |
|  | 2026-05-11T09:56:02.894Z | END RequestId: 006d7ac6-d897-4b39-903e-f568e02ce16a                                                                                                                           |
|  | 2026-05-11T09:56:02.894Z | REPORT RequestId: 006d7ac6-d897-4b39-903e-f568e02ce16a Duration: 1.76 ms Billed Duration: 88 ms Memory Size: 256 MB Max Memory Used: 37 MB Init Duration: 85.82 ms            |
|  | 2026-05-11T09:56:13.899Z | START RequestId: bfe81ee4-a844-47bf-8435-d0b462301fc0 Version: \$LATEST                                                                                                       |
|  | 2026-05-11T09:56:13.899Z | Processed request for: Mario                                                                                                                                                  |
|  | 2026-05-11T09:56:13.899Z | Response: {"message": "Hello, Mario!", "function_name": "hello-serverless", "memory_limit_mb": "256", "remaining_time_ms": 3000, "execution_time_ms": 0.01}                   |
|  | 2026-05-11T09:56:13.901Z | END RequestId: bfe81ee4-a844-47bf-8435-d0b462301fc0                                                                                                                           |
|  | 2026-05-11T09:56:13.901Z | REPORT RequestId: bfe81ee4-a844-47bf-8435-d0b462301fc0 Duration: 1.48 ms Billed Duration: 2 ms Memory Size: 256 MB Max Memory Used: 37 MB                                     |
|  |                          | No newer events at this moment. <i>Auto retry paused.</i> <a href="#">Resume</a>                                                                                              |

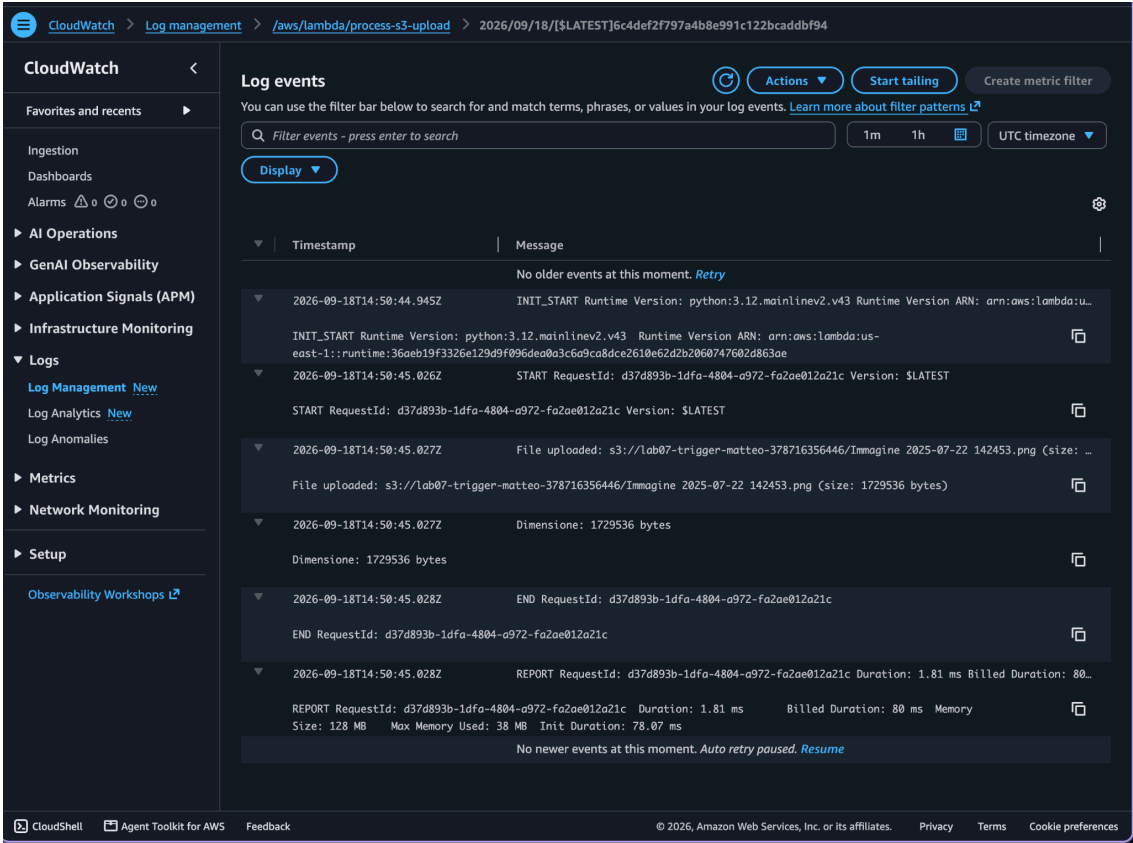
# Lab 07 - Trigger Lambda e Linguaggi

Matteo Ninotti:

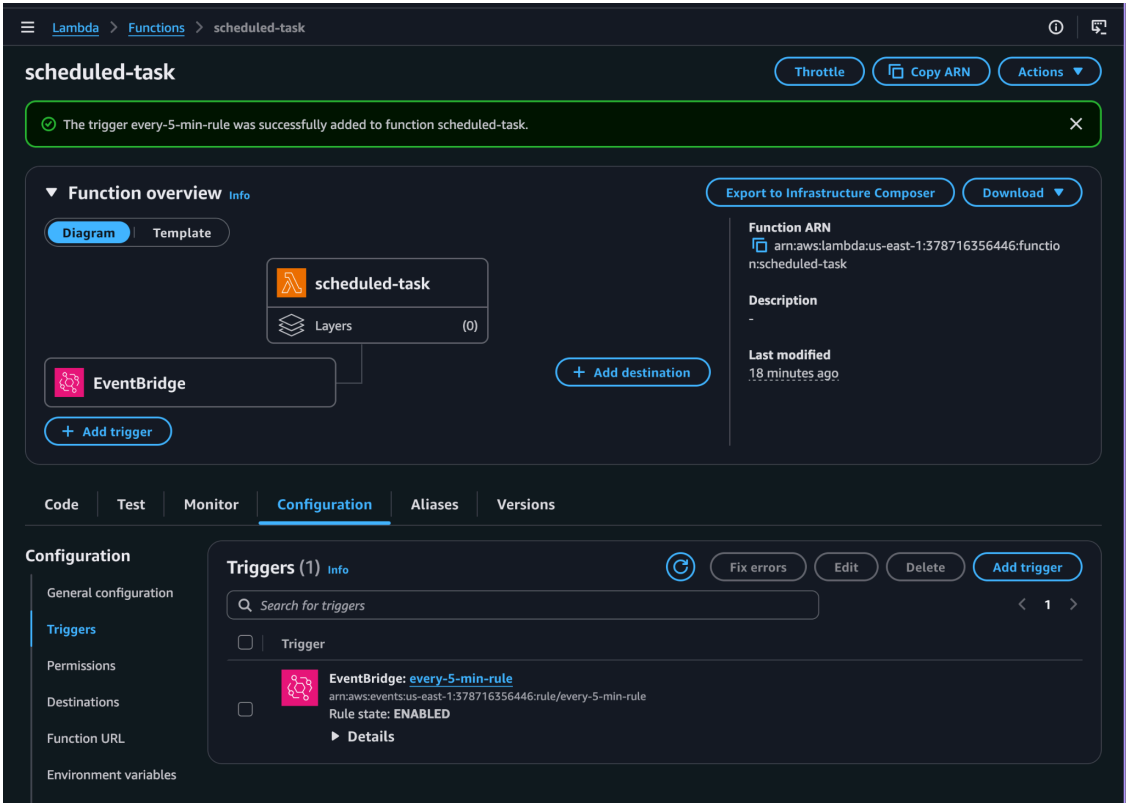
Parte 1 - blob upload event



Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi



Parte 2 - Cron



Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

aws

Search

[Option+S]

United States (N. Virginia)

Account ID: 3787-1635-6446  
voclabs/user4738697=matteo.ninotti@edu-its.it

CloudWatch

Log management

/aws/lambda/scheduled-task

2026/09/18/[LATEST]f51b8723a8b4413290ef3e107c2fe259

CloudWatch

Favorites and recents

Ingestion

Dashboards

Alarms 0 0 0 0

AI Operations

GenAI Observability

Application Signals (APM)

Infrastructure Monitoring

Logs

Log Management New

Log Analytics New

Log Anomalies

Metrics

Network Monitoring

Setup

Observability Workshops

Log events

Actions

Start tailing

Create metric filter

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

1m 1h

UTC timezone

Display

Timestamp

Message

2026-09-18T15:20:24.795Z

START RequestId: 612e272b-aeda-4bf1-91a7-48d1dadd08a7 Version: \$LATEST

2026-09-18T15:20:24.795Z

Scheduled execution at 2026-09-18T15:20:24.795592

2026-09-18T15:20:24.795Z

Event source: {"version": "0", "id": "5ef5857a-41c6-1bc6-d175-f2a72e0d53fb", "detail-type": "Sch...

Event source:

{

"version": "0",

"id": "5ef5857a-41c6-1bc6-d175-f2a72e0d53fb",

"detail-type": "Scheduled Event",

"source": "aws.events",

"account": "378716356446",

"time": "2026-09-18T15:19:50Z",

"region": "us-east-1",

"resources": [

"arn:aws:events:us-east-1:378716356446:rule/every-5-min-rule"

],

"detail": {}

}

2026-09-18T15:20:24.797Z

END RequestId: 612e272b-aeda-4bf1-91a7-48d1dadd08a7

2026-09-18T15:20:24.797Z

REPORT RequestId: 612e272b-aeda-4bf1-91a7-48d1dadd08a7 Duration: 2.07 ms Billed Duration: 88 ms ...

2026-09-18T15:25:24.784Z

START RequestId: e7bd0789-47b6-4f27-af72-75f0f7ffa1fc Version: \$LATEST

2026-09-18T15:25:24.785Z

Scheduled execution at 2026-09-18T15:25:24.785079

2026-09-18T15:25:24.785Z

Event source: {"version": "0", "id": "dc4f31fc-0b8f-3aa4-1dc7-b4feeac9bd43", "detail-type": "Sch...

Event source:

{

"version": "0",

"id": "dc4f31fc-0b8f-3aa4-1dc7-b4feeac9bd43",

"detail-type": "Scheduled Event",

"source": "aws.events",

"account": "378716356446",

"time": "2026-09-18T15:24:50Z",

"region": "us-east-1",

"resources": [

"arn:aws:events:us-east-1:378716356446:rule/every-5-min-rule"

],

"detail": {}

}

2026-09-18T15:25:24.792Z

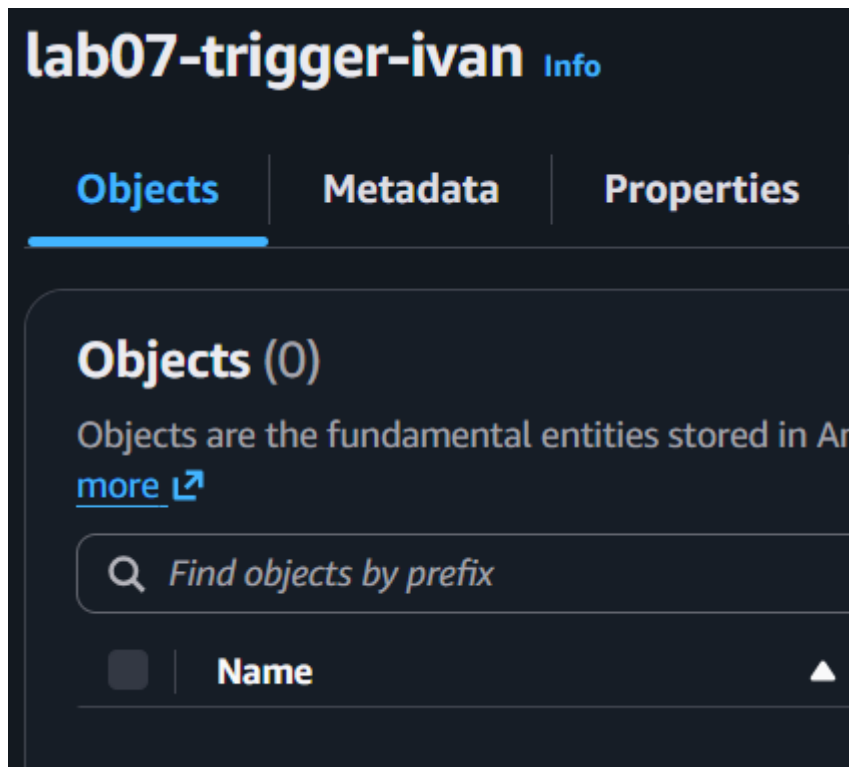
END RequestId: e7bd0789-47b6-4f27-af72-75f0f7ffa1fc

Back to top

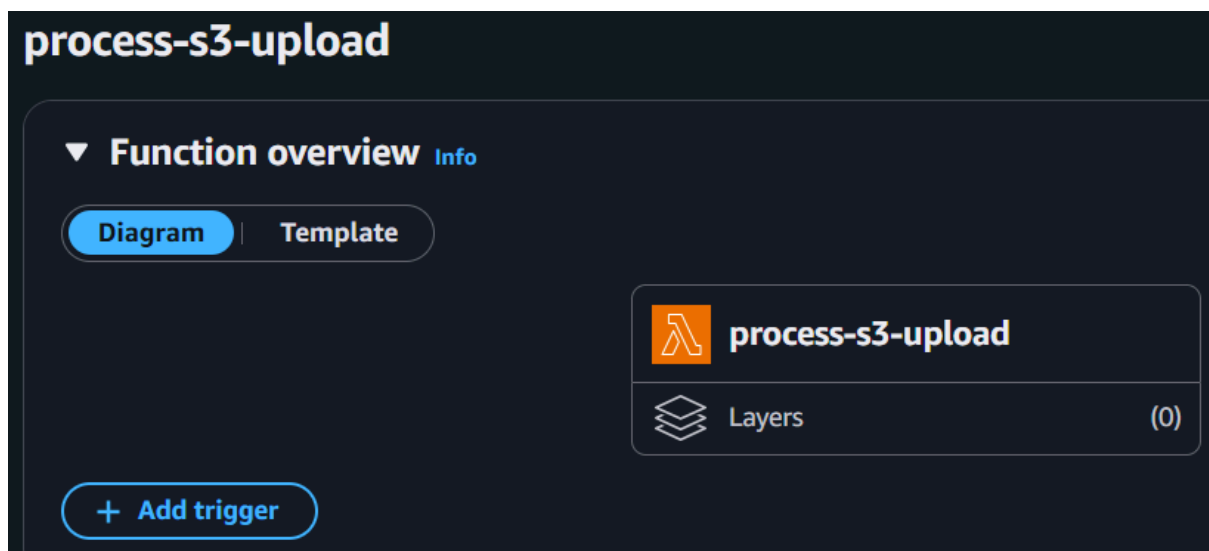
## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

Ivan Cioffi:

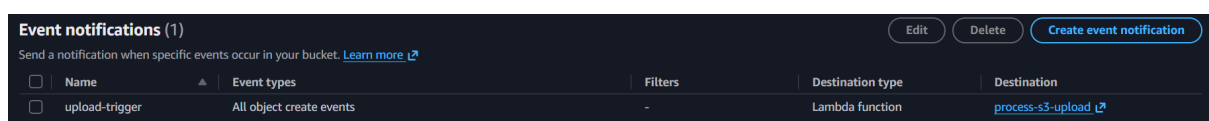
Step 1



Step 2



Step 3



## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

### Step 4

| ▶ | Timestamp                | Message                                                                                                                                  |
|---|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
|   |                          | No older events at this moment. <a href="#">Retry</a>                                                                                    |
| ▶ | 2026-09-20T11:46:56.223Z | INIT_START Runtime Version: python:3.12.mainlinev2.v43 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:36aeb19f3326e129d9f096d... |
| ▶ | 2026-09-20T11:46:56.315Z | START RequestId: 981c902e-6654-48dc-a45d-8fb0964e6f24 Version: \$LATEST                                                                  |
| ▶ | 2026-09-20T11:46:56.316Z | File uploaded: s3://lab07-trigger-ivan/prova lab07.txt (size: 32 bytes)                                                                  |
| ▶ | 2026-09-20T11:46:56.316Z | Dimensione: 32 bytes                                                                                                                     |
| ▶ | 2026-09-20T11:46:56.317Z | END RequestId: 981c902e-6654-48dc-a45d-8fb0964e6f24                                                                                      |
| ▶ | 2026-09-20T11:46:56.317Z | REPORT RequestId: 981c902e-6654-48dc-a45d-8fb0964e6f24 Duration: 1.81 ms Billed Duration: 91 ms Memory Size: 128 MB Max Memory Used: ... |
|   |                          | No newer events at this moment. <i>Auto retry paused.</i> <a href="#">Resume</a>                                                         |

### Step 5

## scheduled-task

▼ Function overview [Info](#)

Diagram

Template

 **scheduled-task**

 Layers (0)

+ Add trigger

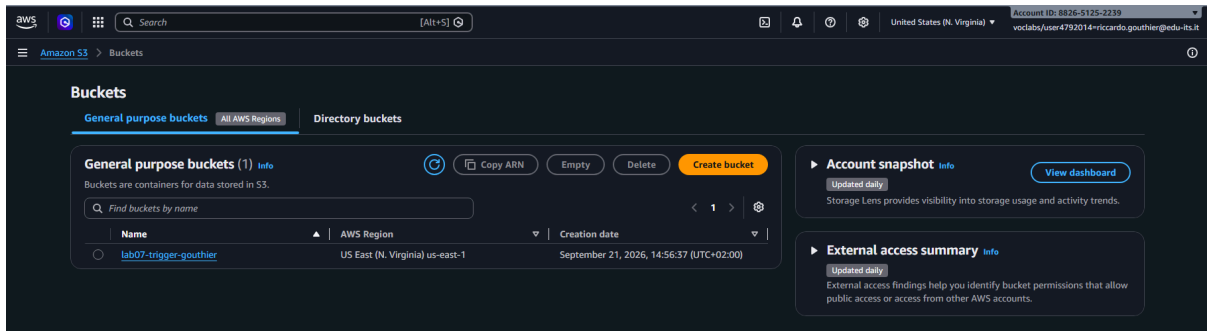
### Step 6

| ▶ | Timestamp                | Message                                                                                                                                |
|---|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|   |                          | No older events at this moment. <a href="#">Retry</a>                                                                                  |
| ▶ | 2026-09-20T11:53:21.877Z | INIT_START Runtime Version: python:3.12.mainlinev2.v43 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:36aeb19f3326e129d9f09... |
| ▶ | 2026-09-20T11:53:21.962Z | START RequestId: 0c6859eb-09a3-461e-8cf4-ef5be80b21bb Version: \$LATEST                                                                |
| ▶ | 2026-09-20T11:53:21.963Z | Scheduled execution at 2026-09-20T11:53:21.963077                                                                                      |
| ▶ | 2026-09-20T11:53:21.963Z | Event source: {"version": "0", "id": "2dd956c5-da0b-b4bd-c4aa-068fcd23c2f0", "detail-type": "Scheduled Event", "source": "aws.event... |
| ▶ | 2026-09-20T11:53:21.965Z | END RequestId: 0c6859eb-09a3-461e-8cf4-ef5be80b21bb                                                                                    |
| ▶ | 2026-09-20T11:53:21.965Z | REPORT RequestId: 0c6859eb-09a3-461e-8cf4-ef5be80b21bb Duration: 2.22 ms Billed Duration: 85 ms Memory Size: 128 MB Max Memory Used... |
| ▶ | 2026-09-20T11:58:21.759Z | START RequestId: 689709d8-4f87-4e25-8465-abe8cfca1283 Version: \$LATEST                                                                |
| ▶ | 2026-09-20T11:58:21.759Z | Scheduled execution at 2026-09-20T11:58:21.759691                                                                                      |
| ▶ | 2026-09-20T11:58:21.759Z | Event source: {"version": "0", "id": "d52e97d0-082b-e336-7cc8-61793c39d56c", "detail-type": "Scheduled Event", "source": "aws.event... |
| ▶ | 2026-09-20T11:58:21.763Z | END RequestId: 689709d8-4f87-4e25-8465-abe8cfca1283                                                                                    |

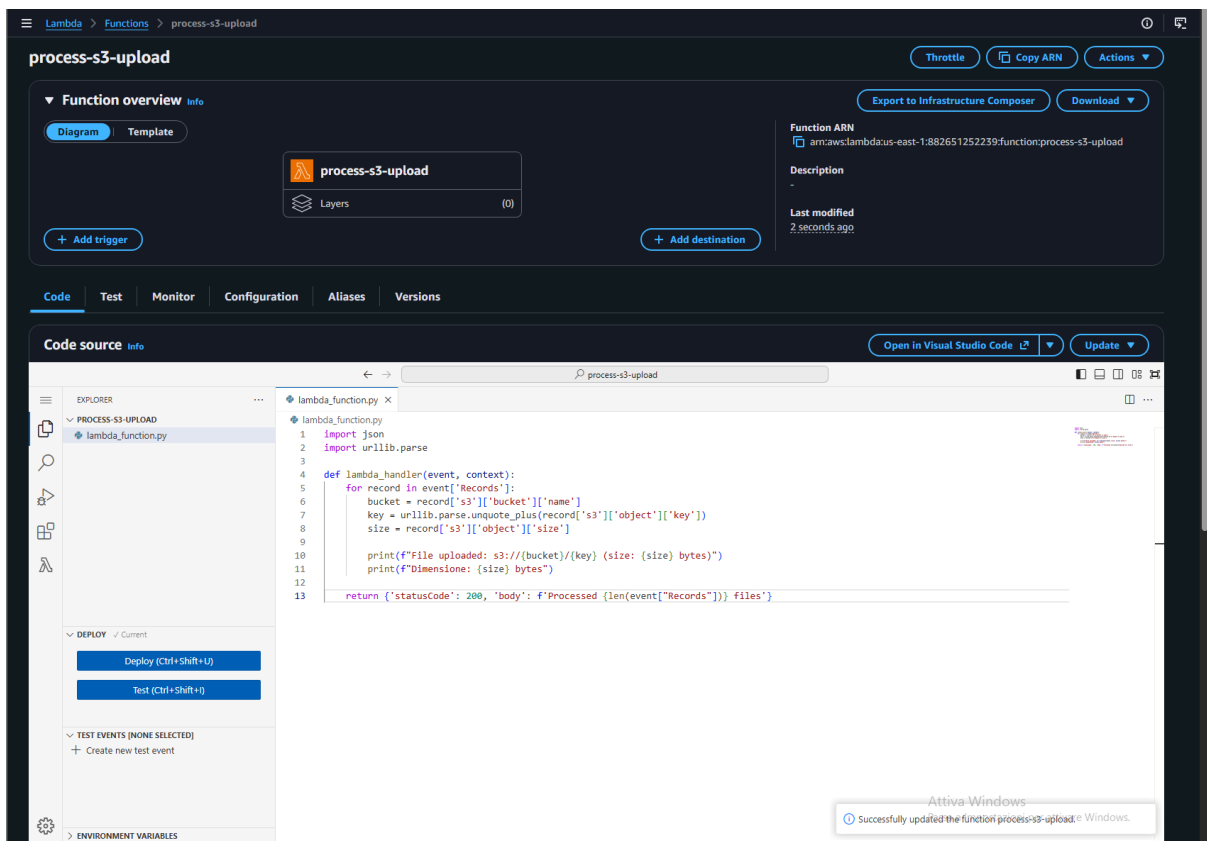
## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

Riccardo Gouthier:

Step 1:

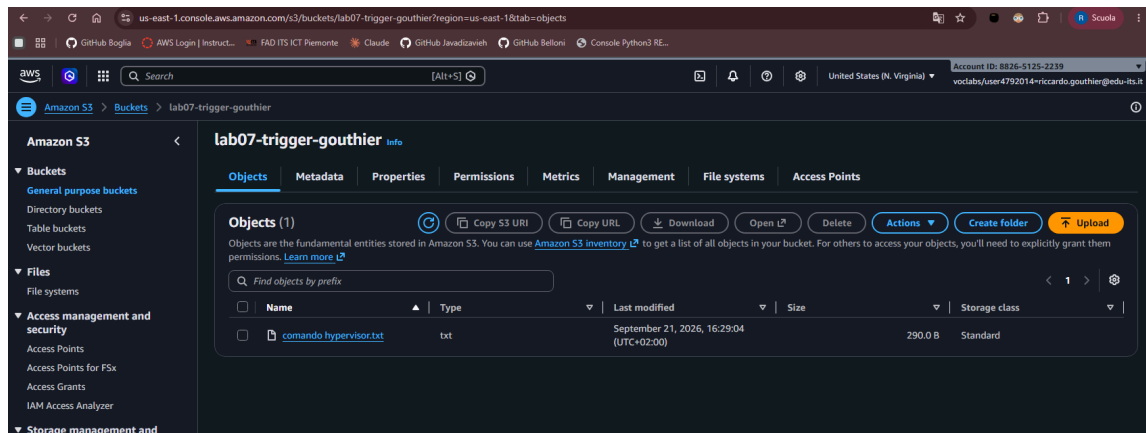


Step 2:

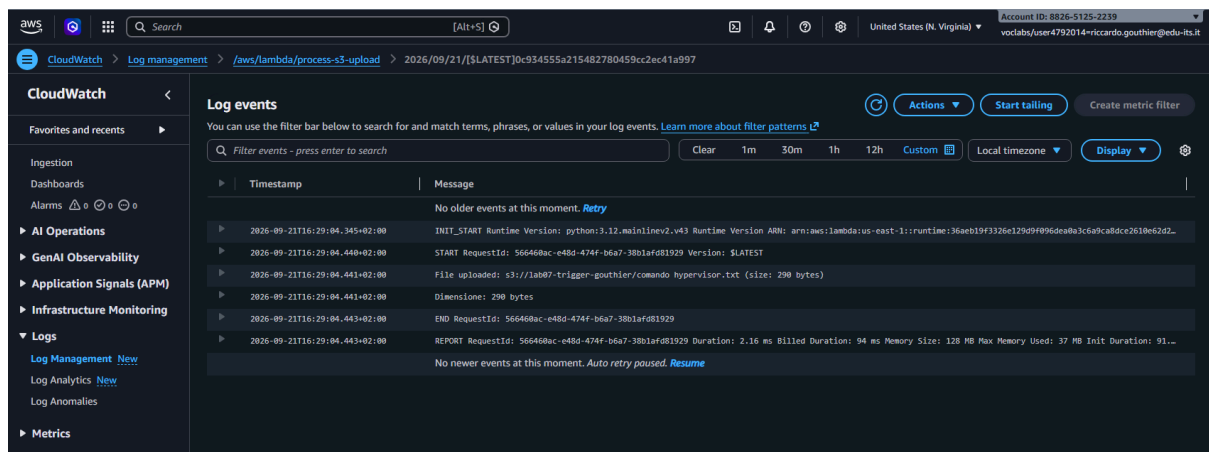


## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

### Step 3:



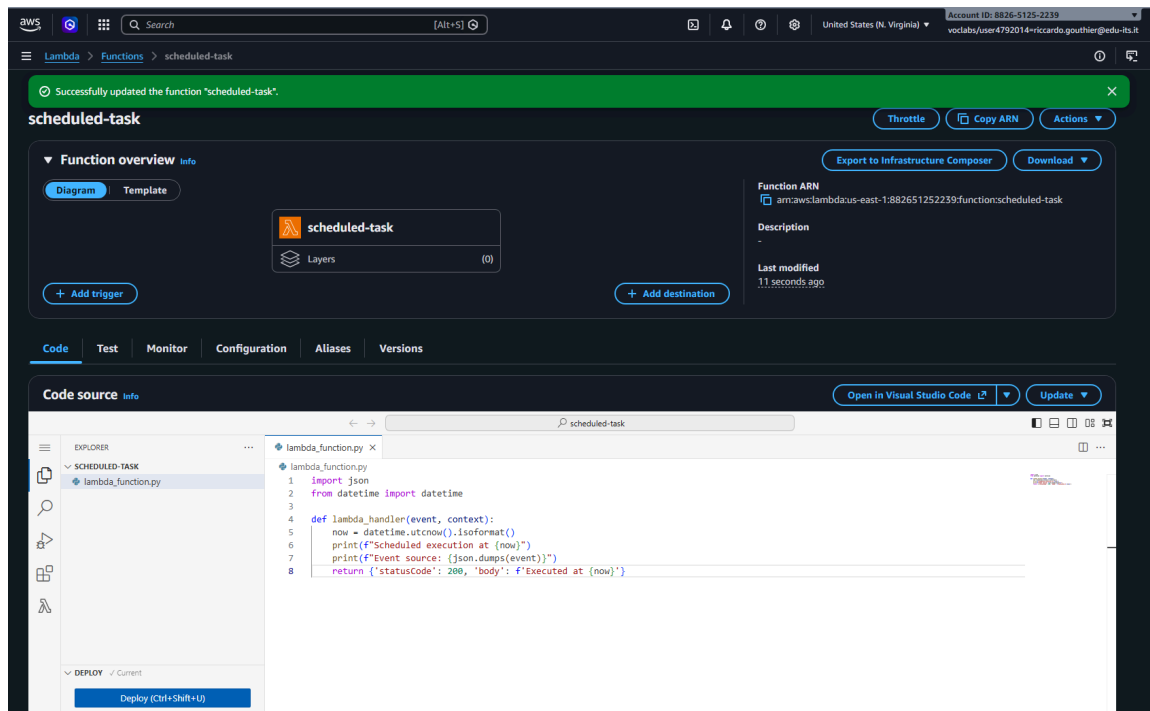
### Step 4:



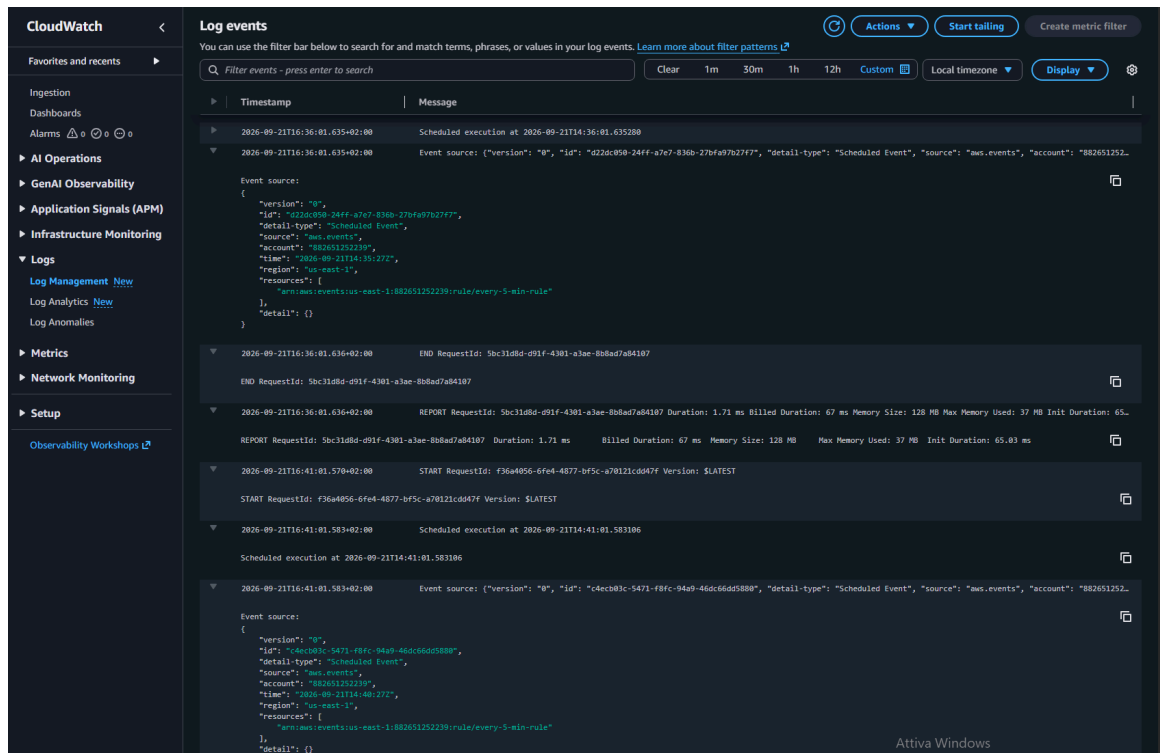


## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

### Step 5:



### Step 6:



# Lab 08-09 - Risorse Lambda, Monitoraggio e Sicurezza IAM

Matteo Ninotti:  
parte A (lab 8)

| Memoria | Durata (ms) | (MB-s)                         |
|---------|-------------|--------------------------------|
| 128 MB  | 1864.14     | = 128 x durata_s =<br>243 MB-s |
| 256 MB  | 983.67      | = 256 x durata_s =<br>252 MB-s |
| 512 MB  | 463.49      | = 512 x durata_s =<br>237 MB-s |
| 1024 MB | 219.64      | = 1024 x durata_s =<br>224MB-s |

resource-test

Throttle

Copy ARN

Actions

Function overview

Diagram

Template

resource-test

Layers

(0)

Add trigger

Add destination

Function ARN

arn:aws:lambda:us-east-1:378716356446:function:resource-test

Description

-

Last modified

1 minute ago

Export to Infrastructure Composer

Download

Code

Test

Monitor

Configuration

Aliases

Versions

Executing function: succeeded

Details

Response

Format JSON

```
{
  "statusCode": 200,
  "body": {
    "duration_ms": 219.64,
    "memory_mb": "1024",
    "result": "666666166.4588221"
  }
}
```

Summary

Code SHA-256

455oBZmbQmXkj3dL2t51Agecv/y/Xooo2DabirbzwVg=

Function version

\$LATEST

Duration

222.00 ms

Resources configured

Execution time

31 seconds ago

Request ID

becbafad-36fa-4fe1-a5c0-2afb5f622405

Billed duration

327 ms

Max memory used



Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

Executing function: succeeded (logs)

Details

Response

```
{
  "statusCode": 404,
  "body": "{\"error\": \"User not found\"}"
}
```

Summary

Code SHA-256

QA1B3udL+OV8CjKJnGi9EcnHUvkpjSFFFoxGEZxOI=

Function version

\$LATEST

Duration

61.52 ms

Resources configured

128 MB

Execution time

10 seconds ago

Request ID

58f23899-f522-45f8-bde1-76a7f11a6d53

Billed duration

62 ms

Max memory used

95 MB

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestId: 58f23899-f522-45f8-bde1-76a7f11a6d53 Version: $LATEST
END RequestId: 58f23899-f522-45f8-bde1-76a7f11a6d53
REPORT RequestId: 58f23899-f522-45f8-bde1-76a7f11a6d53  Duration: 61.52 ms    Billed Duration: 62 ms  Memory Size: 128 MB    Max Memory Used: 95 MB
```

Code

Test

Monitor

Configuration

Aliases

Versions

Executing function: succeeded (logs)

Details

Response

```
{
  "statusCode": 400,
  "body": "{\"error\": \"user_id is required\"}"
}
```

Summary

Code SHA-256

QA1B3udL+OV8CjKJnGi9EcnHUvkpjSFFFoxGEZxOI=

Function version

\$LATEST

Duration

12.73 ms

Resources configured

128 MB

Execution time

30 seconds ago

Request ID

9d55c84e-203f-4cbf-b6c8-9630a9d1dfee

Billed duration

13 ms

Max memory used

95 MB

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestId: 9d55c84e-203f-4cbf-b6c8-9630a9d1dfee Version: $LATEST
END RequestId: 9d55c84e-203f-4cbf-b6c8-9630a9d1dfee
REPORT RequestId: 9d55c84e-203f-4cbf-b6c8-9630a9d1dfee  Duration: 12.73 ms    Billed Duration: 13 ms  Memory Size: 128 MB    Max Memory Used: 95 MB
```

Code

Test

Monitor

Configuration

Aliases

Versions

Executing function: succeeded (logs)

Details

Response

```
{
  "statusCode": 400,
  "body": "{\"error\": \"Invalid user_id format\"}"
}
```

Summary

Code SHA-256

QA1B3udL+OV8CjKJnGi9EcnHUvkpjSFFFoxGEZxOI=

Function version

\$LATEST

Duration

17.80 ms

Resources configured

128 MB

Execution time

2 minutes ago

Request ID

9794b1c0-8c8e-4bdc-90e9-55d5d56ac910

Billed duration

18 ms

Max memory used

95 MB

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestId: 9794b1c0-8c8e-4bdc-90e9-55d5d56ac910 Version: $LATEST
END RequestId: 9794b1c0-8c8e-4bdc-90e9-55d5d56ac910
REPORT RequestId: 9794b1c0-8c8e-4bdc-90e9-55d5d56ac910  Duration: 17.80 ms    Billed Duration: 18 ms  Memory Size: 128 MB    Max Memory Used: 95 MB
```

**Gruppo 2:** *Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi*

permessi minimi per least-privilege

dynamodb:GetItem sulla tabella lab09-users

secretsmanager:GetSecretValue sul secret lab09/app-config

logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents

La policy di LabRole utilizzata ha permessi molto piu' ampi

Ivan Cioffi:

Step 1

resource-test

▼ Function overview Info

Diagram

Template

 resource-test

 Layers (0)

+ Add trigger

Step 2

128 MB

START RequestId: 7b8a66ba-972e-4a26-b41b-47cb8dcblde8 Version: \$LATEST  
{\"iterations\": 1000000, \"duration\_ms\": 1846.1, \"memory\_mb\": \"128\", \"remaining\_ms\": 1153}  
END RequestId: 7b8a66ba-972e-4a26-b41b-47cb8dcblde8  
REPORT RequestId: 7b8a66ba-972e-4a26-b41b-47cb8dcblde8 Duration: 1984.64 ms Billed Duration: 1990 ms Memory Size: 128 MB Max Memory Used: 37 MB Init Duration: 84.76 ms

256 MB

START RequestId: 3945ee95-5320-478b-94bf-4d6caa70d2d1 Version: \$LATEST  
{\"iterations\": 1000000, \"duration\_ms\": 868.07, \"memory\_mb\": \"256\", \"remaining\_ms\": 2131}  
END RequestId: 3945ee95-5320-478b-94bf-4d6caa70d2d1  
REPORT RequestId: 3945ee95-5320-478b-94bf-4d6caa70d2d1 Duration: 986.57 ms Billed Duration: 988 ms Memory Size: 256 MB Max Memory Used: 37 MB Init Duration: 81.42 ms

512 MB

START RequestId: 9529d607-97d4-49a9-9a42-4f32ff4b80c1 Version: \$LATEST  
{\"iterations\": 1000000, \"duration\_ms\": 436.98, \"memory\_mb\": \"512\", \"remaining\_ms\": 2562}  
END RequestId: 9529d607-97d4-49a9-9a42-4f32ff4b80c1  
REPORT RequestId: 9529d607-97d4-49a9-9a42-4f32ff4b80c1 Duration: 439.13 ms Billed Duration: 519 ms Memory Size: 512 MB Max Memory Used: 37 MB Init Duration: 79.48 ms

1024 MB

START RequestId: 2ec5432b-b5ff-4d59-b7af-eb05c0f1fff Version: \$LATEST  
{\"iterations\": 1000000, \"duration\_ms\": 215.54, \"memory\_mb\": \"1024\", \"remaining\_ms\": 2784}  
END RequestId: 2ec5432b-b5ff-4d59-b7af-eb05c0f1fff  
REPORT RequestId: 2ec5432b-b5ff-4d59-b7af-eb05c0f1fff Duration: 228.91 ms Billed Duration: 315 ms Memory Size: 1024 MB Max Memory Used: 37 MB Init Duration: 85.66 ms

| Memoria | Durata (ms) | Costo stimato (MB-s) |
|---------|-------------|----------------------|
| 128 MB  | 1094        | 244                  |
| 256 MB  | 906         | 232                  |
| 512 MB  | 439         | 225                  |

Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

1024 MB      229      234

Step 3

Concurrency

Function concurrency

Use reserved concurrency

Reserved concurrency

5

Step 4

Alarms (1)

Clear selection

Create composite alarm

Actions

Create alarm

Filter alarms

Quick filters

Name

State

Actions

Actions muted

Last state update (UTC)

Conditions

resource-test-errors

Insufficient data

No actions

-

2026-09-20 17:14:23

Errors > 0 for 1 datapoints within 1 minute

Step 5

1 SOURCE logGroups(namePrefix: [], class: "STANDARD") START=-1h END=0s |

2 filter @type = "REPORT"

3 | stats avg(@duration) as avgDuration, max(@duration) as maxDuration, min(@memorySize) as memorySize

Visualize

Discovered fields (11)

Lookup tables

complete: Showing 1 of 5 matched. Query executed for 4 log groups.

# Number

Line

Stacked area

Bar

Pie

Heatmap

View data

879

1.9 K

128 M

avgDuration

maxDuration

memorySize

PARTE B

Step 6

Tables (1) Info

Find tables

Name

Status

Partition key

lab09-users

Active

user\_id (S)

## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

### Step 7

| <input type="checkbox"/> | user_id (String) ▾   | email ▾      | name  |
|--------------------------|----------------------|--------------|-------|
| <input type="checkbox"/> | <a href="#">u002</a> | bob@exam...  | Bob   |
| <input type="checkbox"/> | <a href="#">u001</a> | alice@exa... | Alice |

### Step 8

## secure-reader

▼ Function overview [Info](#)

Diagram

Template

 **secure-reader**

 Layers (0)

+ Add trigger

### Step 9

### Environment variables

You can define environment variables as key-value pairs that are accessible from your function code. These are useful to store configuration settings without t

| Key                                          | Value                                         |
|----------------------------------------------|-----------------------------------------------|
| <input type="text" value="APP_SECRET_NAME"/> | <input type="text" value="lab09/app-config"/> |
| <input type="text" value="TABEL_NAME"/>      | <input type="text" value="lab09-users"/>      |

### Step 10

## Secrets

Secret name

[lab09/app-config](#)



## Step 11

```
{
  "statusCode": 200,
  "body": "{\"user\": {\"email\": \"alice@example.com\", \"name\": \"Alice\", \"user_id\": \"u001\"}, \"secret_loaded\": true,
}
```

### Response

```
{
  "statusCode": 404,
  "body": "{\"error\": \"User not found\"}"
}
```

### Response

```
{
  "statusCode": 400,
  "body": "{\"error\": \"user_id is required\"}"
}
```

### Response

```
{
  "statusCode": 400,
  "body": "{\"error\": \"Invalid user_id format\"}"
}
```

## Step 12

**dynamodb:GetItem** sulla tabella lab09-users

**secretsmanager:GetSecretValue** sul secret lab09/app-config

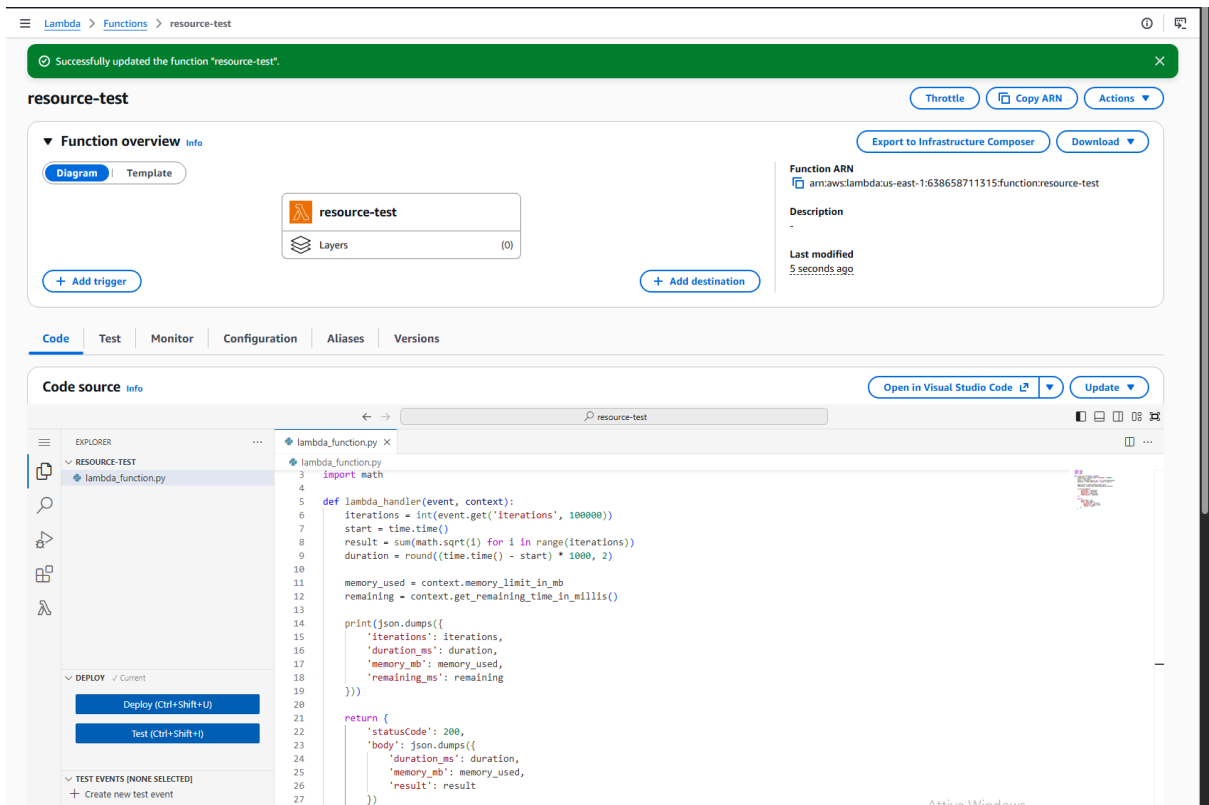
**logs:CreateLogGroup**, **logs:CreateLogStream**, **logs:PutLogEvents**

La policy di LabRole ha permessi molto ampi

Riccardo Gouthier:

PARTE A

Step 1

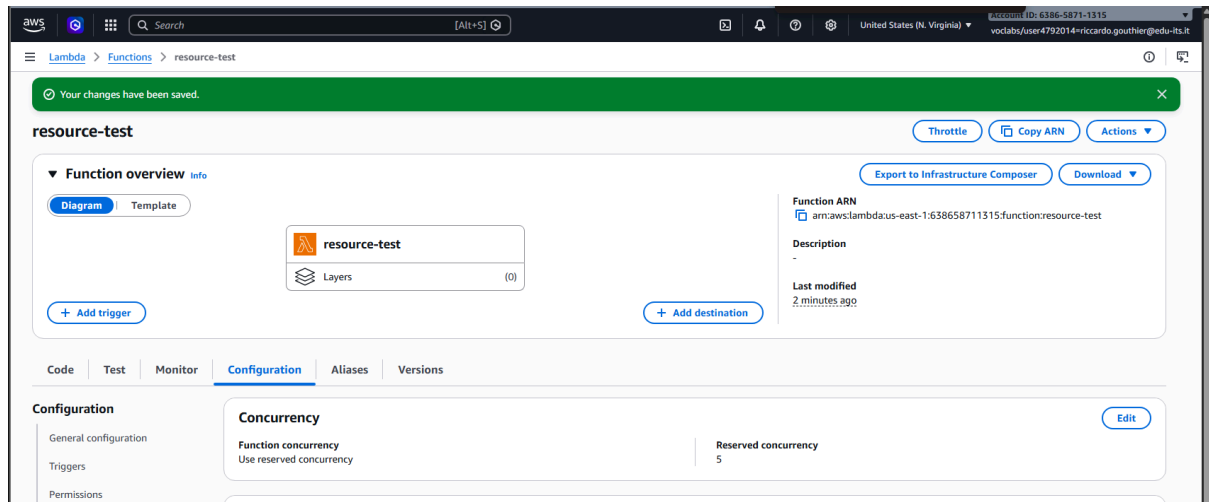


Step 2

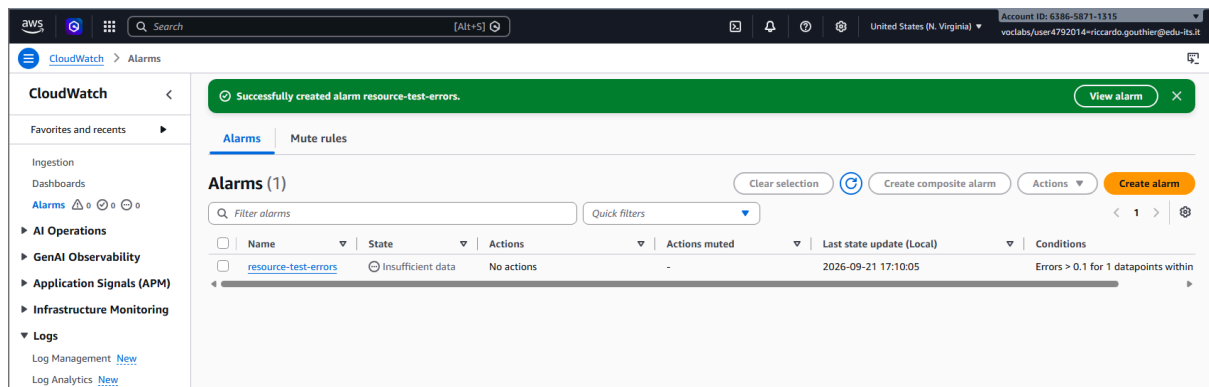
| Memoria | Durata (ms) | Costo stimato (MB-s) |
|---------|-------------|----------------------|
| 128 MB  | 158.11      | = 128 x durata_s     |
| 256 MB  | 66.07       | = 256 x durata_s     |
| 512 MB  | 33.42       | = 512 x durata_s     |
| 1024 MB | 12.83       | = 1024 x durata_s    |

## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

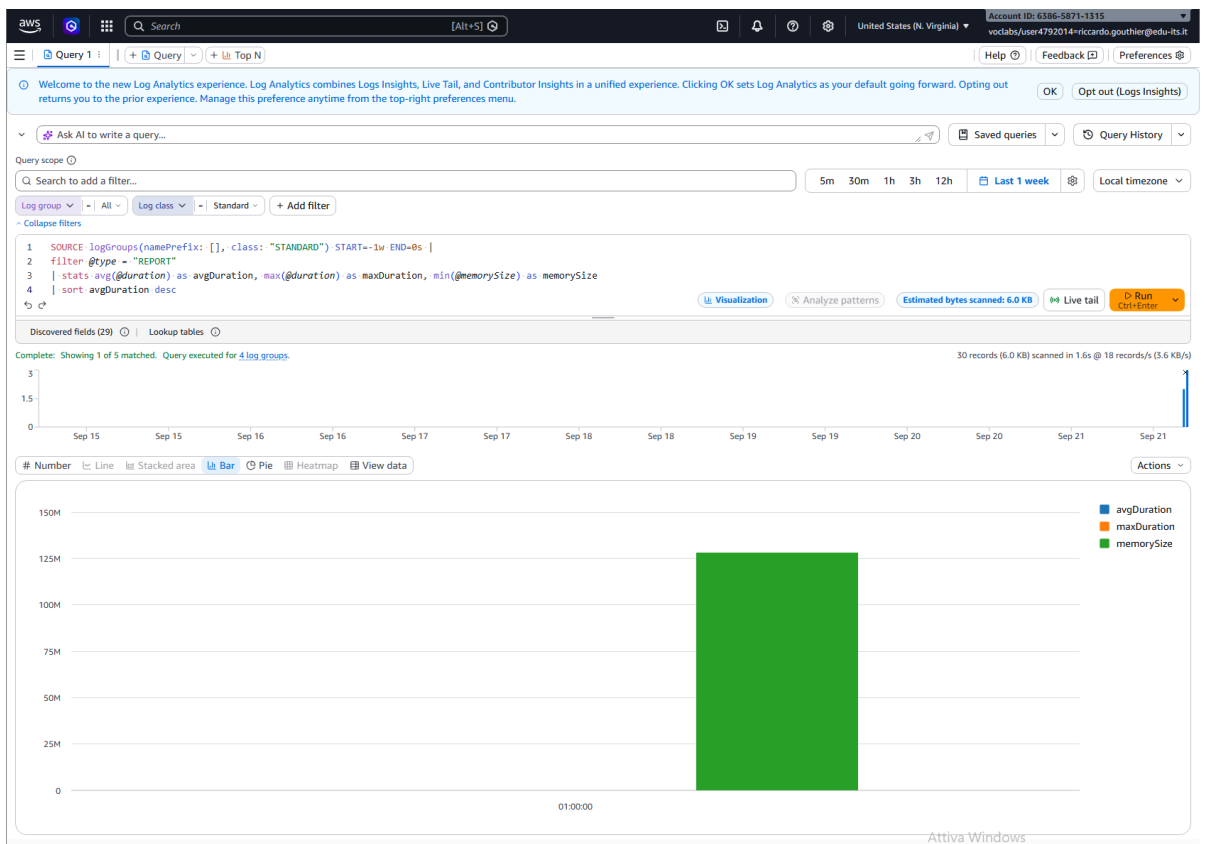
### Step 3



### Step 4



Step 5



PARTE B

Step 6

The screenshot shows the AWS DynamoDB console. A notification at the top states: "The lab09-users table was created successfully." Below, the "Tables (1)" section is displayed. The table "lab09-users" is listed with the following details:

| Name        | Status | Partition key | Sort key | Indexes | Replication Regions | Deletion protection | Favorite | Read capacity mode | Write capacity mode |
|-------------|--------|---------------|----------|---------|---------------------|---------------------|----------|--------------------|---------------------|
| lab09-users | Active | user_id (S)   | -        | 0       | 0                   | Off                 | ☆        | On-demand          | On-demand           |

## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

### Step 7

The screenshot shows the AWS DynamoDB console interface. A green notification bar at the top states "The item has been saved successfully." The left sidebar shows the navigation menu with "DynamoDB" selected. The main content area displays the "lab09-users" table. A "Scan" operation has been completed, returning 2 items. The table structure and the returned items are shown below.

**Tables (1)**

- Filter by tag: Any tag key
- Filter by tag value: Any tag value
- Find tables
- lab09-users

**lab09-users**

▼ Scan, query or search items

Scan Query

Select a table or index: Table - lab09-users

Select attribute projection: All attributes

Filters - optional

Run Reset

Completed - Items returned: 0 - Items scanned: 0 - Efficiency: 100% - RCUs consumed: 2

**Table: lab09-users - Items returned (2)**

Scan started on September 21, 2026, 17:19:16

| user_id (String) | email        | name  |
|------------------|--------------|-------|
| u002             | bob@exam...  | Bob   |
| u001             | alice@exa... | Alice |

### Step 8

The screenshot shows the AWS Lambda console interface for the "secure-reader" function. A green notification bar at the top states "Successfully updated the function 'secure-reader'." The left sidebar shows the navigation menu with "Lambda" selected. The main content area displays the "Code source" tab for the "secure-reader" function. The code source is a Python file named "lambda\_function.py" containing a "lambda\_handler" function. The function prints a success message and returns a 200 status code with a JSON body containing user information. It also includes an exception handler that prints an error message and returns a 500 status code with a JSON body containing the error message.

Successfully updated the function "secure-reader".

Code Test Monitor Configuration Aliases Versions

Code source

Open in Visual Studio Code Update

EXPLORER

- SECURE-READER
- lambda\_function.py

```
17 def lambda_handler(event, context):
33     try:
45         print(json.dumps({
48             'status': 'success',
49             'secret_loaded': True
50         }))
51
52     return {
53         'statusCode': 200,
54         'body': json.dumps({
55             'user': item,
56             'secret_loaded': True,
57             'secret_key_present': 'demo_token' in secret
58         })
59     }
60 except Exception as e:
61     print(json.dumps({
62         'action': 'get_user',
63         'user_id': user_id,
64         'status': 'error',
65         'error': str(e)
66     }))
67     return {
68         'statusCode': 500,
69         'body': json.dumps({'error': 'Internal server error'})
70     }
```

Deploy (Ctrl+Shift+U) Test (Ctrl+Shift+I)

TEST EVENTS [NONE SELECTED] Create new test event

ENVIRONMENT VARIABLES

Lambda Deployed 0 0 Amazon Q

Ln 70, Col 10 Spaces: 4 UTF-8 LF Python Lambda Layout: US

## Gruppo 2: Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi

### Step 9

The screenshot shows the AWS Lambda console for the 'secure-reader' function. A green notification bar at the top states 'Successfully updated the function "secure-reader"'. The 'Function overview' section shows the function name 'secure-reader' and a 'Layers' section with '(0)' layers. The 'Configuration' tab is selected, showing 'Environment variables (2)'. The environment variables are:

| Key             | Value            |
|-----------------|------------------|
| APP_SECRET_NAME | lab09/app-config |
| TABLE_NAME      | lab09-users      |

The 'Function overview' section also displays the 'Function ARN' as 'arn:aws:lambda:us-east-1:638658711315:function:secure-reader' and the 'Last modified' time as '1 minute ago'.

### Step 10

The screenshot shows the AWS Secrets Manager console. A green notification bar at the top states 'You successfully stored the secret lab09/app-config. To show it in the list, choose Refresh. Use the sample code to update your applications to retrieve this secret.' The 'Secrets' section shows a list of secrets with the following details:

| Secret name      | Description | Last retrieved (UTC) | Created on (UTC)               |
|------------------|-------------|----------------------|--------------------------------|
| lab09/app-config | -           | -                    | September 21, 2026 at 15:28:52 |

### Step 11

#### user 001

The screenshot shows the AWS Lambda console for the 'secure-reader' function. The 'Execution Results' tab is selected, showing the following response:

```
{
  "statusCode": 200,
  "body": "{\"user\": {\"email\": \"alice@example.com\", \"name\": \"Alice\", \"user_id\": \"u001\"}, \"secret_loaded\": true, \"secret_key_present\": true}"
}
```

The status is 'Succeeded' and the test event name is 'hello-world'.

#### user 999 (deve dare errore)

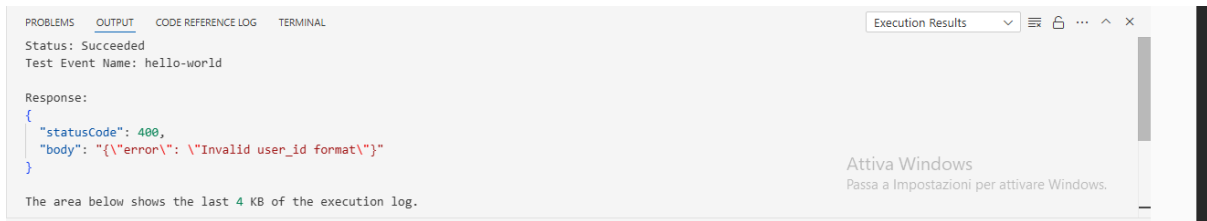
The screenshot shows the AWS Lambda console for the 'secure-reader' function. The 'Execution Results' tab is selected, showing the following response:

```
{
  "statusCode": 404,
  "body": "{\"error\": \"User not found\"}"
}
```

The status is 'Succeeded' and the test event name is 'hello-world'.

## Gruppo 2: *Matteo Ninotti / Riccardo Gouthier / Ivan Cioffi*

**`{"user_id": ""; DROP TABLE users;--"}`**



### Step 12

#### Azioni IAM minime richieste:

1. **dynamodb:GetItem** su tabella lab09-users
2. **secretsmanager:GetSecretValue** su secret lab09/app-config
3. **logs:CreateLogGroup**, **logs:CreateLogStream**, **logs:PutLogEvents**